Video Upload Automation

Ok so with my FakeTube code, I would like to essentially take the video footage on upload of the video itself and use the title uploaded to upload it automatically to the FakeTube youtube channel. I have the auth setup already just not the recurring reupdating of it when the tokens expire but we can deal with that later. I would like to programmatically determine whether the video should be uploaded as a shorts video or a long form one based on length of the video clip and aspect ratio. This can be a bash script that is triggered in the background after upload or what have you but this is the effect I am going for. I would prefer to create entirely new files for this but I can upload my pre-existing ones if you will need them.

Yes — this is a clean setup, and you do **not** need to modify your existing auth flow right away to get the core behavior working.

The simplest approach is:

1. FakeTube saves the uploaded video locally.
2. A background-triggered script runs `ffprobe` to read:
   - duration
   - width
   - height
3. Your worker decides:
   - **Short** if the video is **square or vertical** and **3 minutes or less**
   - **Long-form** otherwise
     YouTube's current Help docs say Shorts can be uploaded at up to **3 minutes** and must be **square or vertical**. The YouTube Data API upload endpoint itself is still just `videos.insert`, so there is no separate "Shorts upload" API switch exposed there; classification is effectively based on the uploaded media/metadata rather than a dedicated Shorts flag. Google Help+2

A wider aspect ratio like 16:9 should be treated as long-form. YouTube's standard player aspect ratio is 16:9, which makes that a sensible default bucket for long-form uploads. Google Help+1

Here's a clean file layout you can add as entirely new files:

```
faketube/
├─ scripts/
│  └─ post_upload.sh
├─ workers/
│  ├─ media_inspector.py
│  ├─ youtube_classifier.py
│  └─ youtube_uploader.py
└─ uploads/
```

1) `workers/media_inspector.py`

This uses `ffprobe` to pull duration, width, and height. `ffprobe` is the right tool for extracting media stream details programmatically. ffmpeg.org+1

```python
#!/usr/bin/env python3

import json
import subprocess
from pathlib import Path


def inspect_video(video_path: str) -> dict:
    path = Path(video_path)
    if not path.exists():
        raise FileNotFoundError(f"Video not found: {video_path}")

    cmd = [
        "ffprobe",
        "-v", "error",
        "-print_format", "json",
        "-show_entries", "format=duration:stream=width,height,codec_type",
        str(path),
    ]

    result = subprocess.run(cmd, capture_output=True, text=True, check=True)
    data = json.loads(result.stdout)

    duration = 0.0
    if "format" in data and "duration" in data["format"]:
        duration = float(data["format"]["duration"])

    width = None
    height = None
    for stream in data.get("streams", []):
        if stream.get("codec_type") == "video":
            width = stream.get("width")
            height = stream.get("height")
            break

    if width is None or height is None:
        raise ValueError(f"Could not determine width/height for: {video_path}")

    aspect_ratio = width / height if height else 0

    return {
        "path": str(path),
        "duration_seconds": duration,
        "width": width,
        "height": height,
        "aspect_ratio": aspect_ratio,
        "is_vertical": height > width,
        "is_square": height == width,
        "is_horizontal": width > height,
    }
```

2) `workers/youtube_classifier.py`

This decides whether it should be uploaded as Shorts-oriented content or long-form.

```python
#!/usr/bin/env python3

from media_inspector import inspect_video


SHORTS_MAX_SECONDS = 180.0  # 3 minutes


def classify_for_youtube(video_path: str) -> dict:
    meta = inspect_video(video_path)

    shorts_candidate = (
        meta["duration_seconds"] <= SHORTS_MAX_SECONDS
        and (meta["is_vertical"] or meta["is_square"])
    )

    upload_type = "shorts" if shorts_candidate else "long"

    return {
        **meta,
        "upload_type": upload_type,
    }


if __name__ == "__main__":
    import json
    import sys

    if len(sys.argv) != 2:
        print("Usage: python youtube_classifier.py <video_path>")
        sys.exit(1)

    result = classify_for_youtube(sys.argv[1])
    print(json.dumps(result, indent=2))
```

3) `workers/youtube_uploader.py`

This assumes your auth setup is already mostly working. The YouTube Data API upload method is **`videos.insert`**. The sample below uses the standard Python Google API client approach described in Google's upload guide. Google for Developers+1

```python
#!/usr/bin/env python3

import argparse
import json
from pathlib import Path

from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

from youtube_classifier import classify_for_youtube

# Replace this with however you already construct your authenticated service
def get_authenticated_service():
    """
    Stub: plug in your existing auth flow here.
    Should return an authenticated youtube service object.
    """
    raise NotImplementedError("Wire this into your existing YouTube auth code.")


def build_description(upload_type: str, base_description: str = "") -> str:
    if upload_type == "shorts":
        if "#Shorts" not in base_description:
            return (base_description.strip() + "\n\n#Shorts").strip()
    return base_description.strip()


def upload_video(
    file_path: str,
    title: str,
    description: str = "",
    privacy_status: str = "public",
    category_id: str = "22",
):
    meta = classify_for_youtube(file_path)
    upload_type = meta["upload_type"]

    final_description = build_description(upload_type, description)

    youtube = get_authenticated_service()

    body = {
        "snippet": {
            "title": title,
            "description": final_description,
            "categoryId": category_id,
        },
        "status": {
            "privacyStatus": privacy_status,
            "selfDeclaredMadeForKids": False,
        },
    }

    media = MediaFileUpload(file_path, chunksize=-1, resumable=True)

    request = youtube.videos().insert(
        part="snippet,status",
        body=body,
        media_body=media,
    )

    response = None
    while response is None:
        status, response = request.next_chunk()
        if status:
            print(f"Upload progress: {int(status.progress() * 100)}%")

    print(json.dumps({
        "youtube_response": response,
        "detected_upload_type": upload_type,
        "duration_seconds": meta["duration_seconds"],
        "width": meta["width"],
        "height": meta["height"],
    }, indent=2))
```

```
    return response


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--file", required=True)
    parser.add_argument("--title", required=True)
    parser.add_argument("--description", default="")
    parser.add_argument("--privacy", default="public")
    parser.add_argument("--category", default="22")
    args = parser.parse_args()

    upload_video(
        file_path=args.file,
        title=args.title,
        description=args.description,
        privacy_status=args.privacy,
        category_id=args.category,
    )
```

4) `scripts/post_upload.sh`

This is the Bash wrapper you can trigger after FakeTube finishes saving the upload.

```
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH="${1:-}"
VIDEO_TITLE="${2:-}"

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" ]]; then
  echo "Usage: ./scripts/post_upload.sh <video_path> <video_title>"
  exit 1
fi

PYTHON_BIN="${PYTHON_BIN:-python3}"

echo "Processing uploaded video..."
"$PYTHON_BIN" workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --description "Uploaded automatically from FakeTube."
```

Make it executable:

```
chmod +x scripts/post_upload.sh
```

5) Example Flask integration

When the upload finishes in your Flask route, call the Bash script in the background:

```python
import os
import subprocess
from pathlib import Path
from flask import Flask, request, jsonify

app = Flask(__name__)
UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(exist_ok=True)

@app.route("/upload", methods=["POST"])
def upload():
    file = request.files["video"]
    title = request.form.get("title", "").strip()

    if not title:
        return jsonify({"error": "Missing title"}), 400

    save_path = UPLOAD_DIR / file.filename
    file.save(save_path)

    subprocess.Popen(
        ["bash", "scripts/post_upload.sh", str(save_path), title],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )

    return jsonify({
        "message": "Upload received and background YouTube processing started.",
        "path": str(save_path),
        "title": title,
    }), 202
```

Why this setup works well

It keeps the logic separated:

- `media_inspector.py` → reads the file
- `youtube_classifier.py` → decides Shorts vs long
- `youtube_uploader.py` → uploads
- `post_upload.sh` → easy trigger point from Flask, cron, CI, or some other worker

That separation will make the later token-refresh work much easier too.

Important practical note

There is not really a “set `is_short=true`” field in the upload API docs. The correct way to think about this is:

- upload through `videos.insert`
- make sure the media qualifies for Shorts if you want Shorts behavior
- optionally append `#Shorts` in the description/title as a signal, though the core classification depends on the media characteristics, not a dedicated API flag. This is consistent with YouTube’s current Help guidance and the API surface. Google Help+2

My recommendation for your FakeTube implementation

Use this rule set:

- **Shorts**:
  - duration `<= 180s`
  - `height >= width`
- **Long-form**:
  - anything else

That is simple, predictable, and aligned with current YouTube behavior. Google Help+1

If you upload your existing auth file or current Flask upload route, I can wire these new files directly to your structure and make the import points exact.

This is my App.jsx file in src/ and I would like all of what you mentioned included with shorts behaviors. I am going to add the shell scripts and python scripts now, just remember this will add much more to ci with python, I would almost prefer to just call a background subprocess with react for one script?

```
import { useState, useRef } from 'react';
import ViralDashboard from './components/ViralDashboard';
import './App.css';
import { Analytics } from "@vercel/analytics/react"


function App() {
  const [videoTitle, setVideoTitle] = useState('');
  const [videoFile, setVideoFile] = useState(null);
  const [started, setStarted] = useState(false);
  const fileInputRef = useRef(null);
  const [views, setViews] = useState(0);
  const [likes, setLikes] = useState(0);
  const [subs, setSubs] = useState(0);
  const [analyticsData, setAnalyticsData] = useState([]);


  const handleFileUpload = (e) => {
    const file = e.target.files[0];
    if (file) {
      setVideoFile(URL.createObjectURL(file));
    }
  };

  const resetSimulation = () => {
    setVideoTitle('');
    setVideoFile(null);
    setStarted(false);
  };

  return (
    <div className="app">
      <header className="header">
        <div className="brand-title">
```

```
          <span className="brand">FakeTube</span>
        </div>

        <a href="https://www.buymeacoffee.com/FakeTube" target="_blank" rel="noopener
noreferrer">
          <img
            src="https://www.buymeacoffee.com/assets/img/guidelines/download-assets-sm-1.svg"
            alt="Buy Me A Coffee"
            style={{ height: '40px', marginTop: '10px' }}
          />
        </a>
        <p className="tagline">World's Totally Not Most Realistic Virality Simulator</p>
      </header>

      <Analytics />
      {!started ? (
        <div className="upload-card">
          <input
            type="text"
            placeholder="Enter fake video title..."
            value={videoTitle}
            onChange={e => setVideoTitle(e.target.value)}
            className="title-input"
          />

          <input
            type="file"
            accept="video/*"
            onChange={handleFileUpload}
            ref={fileInputRef}
            style={{ display: 'none' }}
          />

          <button
            className="yt-button"
            onClick={() => fileInputRef.current.click()}
          >
            📤 Upload Video
          </button>

          <button
            className="yt-button"
            disabled={!videoTitle || !videoFile}
            onClick={() => setStarted(true)}
          >
            ▶️ Start Simulation
          </button>
        </div>
      ) : (
        <ViralDashboard
          title={videoTitle}
```

```
          videoSrc={videoFile}
          onStop={resetSimulation}
          views={views}
          setViews={setViews}
          likes={likes}
          setLikes={setLikes}
          subs={subs}
          setSubs={setSubs}
          analyticsData={analyticsData}
          setAnalyticsData={setAnalyticsData}
        />
      )}
    </div>
  );
}

export default App;
```

Yeah — **do not try to call a background subprocess from React itself**.

Your `App.jsx` is running in the browser, and the browser cannot safely spawn local shell/Python processes. The right setup is:

- **React frontend** uploads the real video file + title to your backend
- **Backend** saves the file
- **Backend** fires **one background subprocess**
- That subprocess can call your shell script, which then calls Python if needed

That gives you the effect you want without stuffing Python logic into the frontend.

For your case, I'd keep it like this:

- `App.jsx` handles:
  - picking the file
  - sending `FormData` to `/api/upload`
  - starting the FakeTube simulation in the UI
- backend handles:
  - saving the uploaded file
  - determining shorts vs long-form
  - uploading to YouTube in the background

So your CI does **not** need React to know anything about Python. Your backend/deploy just needs Python available wherever the upload worker runs.

Updated `App.jsx`

This version keeps your current simulation flow, but instead of only making a blob URL, it also preserves the real file object so it can be posted to your backend.

```
import { useState, useRef } from 'react';
import ViralDashboard from './components/ViralDashboard';
import './App.css';
import { Analytics } from "@vercel/analytics/react";

function App() {
  const [videoTitle, setVideoTitle] = useState('');
  const [videoFile, setVideoFile] = useState(null);          // actual File object
  const [videoPreviewUrl, setVideoPreviewUrl] = useState(null); // blob preview url
  const [started, setStarted] = useState(false);
  const [uploading, setUploading] = useState(false);
  const [uploadMessage, setUploadMessage] = useState('');
  const [youtubeQueued, setYoutubeQueued] = useState(false);
  const [detectedType, setDetectedType] = useState(null);

  const fileInputRef = useRef(null);
  const [views, setViews] = useState(0);
  const [likes, setLikes] = useState(0);
  const [subs, setSubs] = useState(0);
  const [analyticsData, setAnalyticsData] = useState([]);

  const handleFileUpload = (e) => {
    const file = e.target.files[0];
    if (file) {
      setVideoFile(file);

      if (videoPreviewUrl) {
        URL.revokeObjectURL(videoPreviewUrl);
      }

      setVideoPreviewUrl(URL.createObjectURL(file));
      setUploadMessage('');
      setYoutubeQueued(false);
      setDetectedType(null);
    }
  };

  const uploadToBackend = async () => {
    if (!videoFile || !videoTitle.trim()) {
      setUploadMessage('Please provide both a title and a video file.');
      return false;
    }

    try {
      setUploading(true);
      setUploadMessage('Uploading video and queueing YouTube background job...');

      const formData = new FormData();
      formData.append('video', videoFile);
      formData.append('title', videoTitle);

      const response = await fetch('/api/upload', {
        method: 'POST',
        body: formData,
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.error || 'Upload failed');
      }

      setYoutubeQueued(true);
      setDetectedType(data.detected_upload_type || null);
      setUploadMessage(
        data.message || 'Video uploaded successfully and queued for YouTube.'
      );

      return true;
    } catch (err) {
      console.error(err);
      setUploadMessage('Upload failed: ${err.message}');
      return false;
    } finally {
      setUploading(false);
```

```
  }
};

const handleStartSimulation = async () => {
  const ok = await uploadToBackend();
  if (ok) {
    setStarted(true);
  }
};

const resetSimulation = () => {
  setVideoTitle('');
  setVideoFile(null);

  if (videoPreviewUrl) {
    URL.revokeObjectURL(videoPreviewUrl);
  }

  setVideoPreviewUrl(null);
  setStarted(false);
  setUploading(false);
  setUploadMessage('');
  setYoutubeQueued(false);
  setDetectedType(null);
};

return (
  <div className="app">
    <header className="header">
      <div className="brand-title">
        <span className="brand">FakeTube</span>
      </div>

      <a
        href="https://www.buymeacoffee.com/FakeTube"
        target="_blank"
        rel="noopener noreferrer"
      >
        <img
          src="https://www.buymeacoffee.com/assets/img/guidelines/download-assets-sm-1.svg"
          alt="Buy Me A Coffee"
          style={{ height: '40px', marginTop: '10px' }}
        />
      </a>

      <p className="tagline">World’s Totally Not Most Realistic Virality Simulator</p>
    </header>

    <Analytics />

    {!started ? (
      <div className="upload-card">
        <input
          type="text"
          placeholder="Enter fake video title..."
          value={videoTitle}
          onChange={(e) => setVideoTitle(e.target.value)}
          className="title-input"
        />

        <input
          type="file"
          accept="video/*"
          onChange={handleFileUpload}
          ref={fileInputRef}
          style={{ display: 'none' }}
        />

        <button
          className="yt-button"
          onClick={() => fileInputRef.current.click()}
          disabled={uploading}
        >
          📤 Upload Video
        </button>
```

```
          {videoFile && (
            <p style={{ marginTop: '10px' }}>
              Selected: <strong>{videoFile.name}</strong>
            </p>
          )}

          {detectedType && (
            <p style={{ marginTop: '10px' }}>
              Detected YouTube type: <strong>{detectedType}</strong>
            </p>
          )}

          {uploadMessage && (
            <p style={{ marginTop: '10px' }}>
              {uploadMessage}
            </p>
          )}

          <button
            className="yt-button"
            disabled={!videoTitle || !videoFile || uploading}
            onClick={handleStartSimulation}
          >
            {uploading ? '⏳ Uploading...' : '▶ Start Simulation'}
          </button>

          {youtubeQueued && (
            <p style={{ marginTop: '10px' }}>
              ✅ Background YouTube upload job queued.
            </p>
          )}
        </div>
      ) : (
        <ViralDashboard
          title={videoTitle}
          videoSrc={videoPreviewUrl}
          onStop={resetSimulation}
          views={views}
          setViews={setViews}
          likes={likes}
          setLikes={setLikes}
          subs={subs}
          setSubs={setSubs}
          analyticsData={analyticsData}
          setAnalyticsData={setAnalyticsData}
        />
      )}
    </div>
  );
}

export default App;
```

What changed

Your original code only stored:

```
setVideoFile(URL.createObjectURL(file));
```

That is just a browser preview URL. Good for playback, but **not** good for sending the actual file to the backend.

Now you keep both:

- `videoFile` → actual uploaded `File`
- `videoPreviewUrl` → blob URL for the dashboard preview

That lets you:

- simulate locally in the UI
- upload the real file to your backend for YouTube processing

---

Best backend pattern for this

You said you'd rather just call one script in the background. Good. Do that.

Your backend route should do this:

1. receive file + title
2. save file
3. inspect video with Python
4. determine `shorts` vs `long`
5. kick off one subprocess:
    - `bash scripts/post_upload.sh <saved_video_path> <title>`
6. immediately return JSON to React

Example Flask backend route

```python
import os
import uuid
import subprocess
from pathlib import Path
from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename

from workers.youtube_classifier import classify_for_youtube

app = Flask(__name__)

UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(parents=True, exist_ok=True)


@app.route("/api/upload", methods=["POST"])
def upload_video():
    file = request.files.get("video")
    title = request.form.get("title", "").strip()

    if not file:
        return jsonify({"error": "Missing video file"}), 400

    if not title:
        return jsonify({"error": "Missing title"}), 400

    original_name = secure_filename(file.filename or "upload.mp4")
    unique_name = f"{uuid.uuid4().hex} {original_name}"
    save_path = UPLOAD_DIR / unique_name
    file.save(save_path)

    try:
        meta = classify_for_youtube(str(save_path))
    except Exception as e:
        return jsonify({"error": f"Failed to inspect video: {str(e)} }), 500

    subprocess.Popen(
        ["bash", "scripts/post_upload.sh", str(save_path), title],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )

    return jsonify({
        "message": "Upload received. Background YouTube job started.",
        "file_path": str(save_path),
        "detected_upload_type": meta["upload_type"],
        "duration_seconds": meta["duration_seconds"],
        "width": meta["width"],
        "height": meta["height"],
    }), 202
```

Example `scripts/post_upload.sh`

```bash
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH= ${1:-}
VIDEO_TITLE= ${2:-}

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" ]]; then
  echo "Usage: ./scripts/post_upload.sh <video_path> <video_title>"
  exit 1
fi

python3 workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --description "Uploaded automatically from FakeTube."
```

Shorts behavior rule

I'd keep it simple and deterministic:

```
SHORTS_MAX_SECONDS = 180.0

shorts_candidate = (
    duration_seconds <= SHORTS_MAX_SECONDS and height >= width
)
```

So:

- `<= 3 minutes`
- vertical or square
- then treat as Shorts-oriented upload

Everything else goes long-form.

Why this is better than "React subprocess"

Because this:

```
subprocess.Popen(...)
```

belongs on the **server**, not in the browser.

React can only do stuff like:

- `fetch(...)`
- manage state
- display preview/video/player/UI

It cannot safely launch:

- bash scripts
- Python
- ffprobe
- YouTube uploader scripts

So the clean mental model is:

- **React = trigger**
- **Flask/Node backend = worker launcher**
- **shell/Python = media pipeline**

CI impact

This really only adds Python requirements to the environment running the backend worker. That usually means:

- `python3`
- `ffmpeg` / `ffprobe`
- your Python deps
- Google API libs

Your frontend CI can stay mostly normal.

If you are deploying the whole app together, then yes, your deployment image needs both:

- Node for frontend build
- Python + ffmpeg for upload worker behavior

But that is still way cleaner than trying to force this into frontend-only logic.

Recommended file layout

```
src/
  App.jsx
components/
  ViralDashboard.jsx

scripts/
  post_upload.sh

workers/
  media_inspector.py
  youtube_classifier.py
  youtube_uploader.py

uploads/
```

My recommendation

Use your updated `App.jsx` exactly like above, and let React call:

```
fetch('/api/upload', { method: 'POST', body: formData })
```

Then let the backend fire your single shell script in the background.

That gives you:

- local FakeTube preview
- real uploaded file handling
- automatic YouTube upload
- automatic shorts/long classification
- minimal frontend complexity

Paste your backend file next and I'll wire the exact route and script calls into your real structure.

> Ok well I have all the scripts added both bash and python and App.jsx updated, next I'll add the flask backend route? How should we do this?

Yes — add the Flask backend route next.

The cleanest setup is:

- React sends `multipart/form-data` with the real video file and title.
- Flask reads the uploaded file from `request.files`, saves it to disk, and returns a quick JSON response. Flask's documented upload pattern is exactly this flow: get the file from `request.files`, then call `.save()` on it. Flask Documentation+1
- After saving, Flask launches your background worker with `subprocess.Popen(...)`. Python's `subprocess` module is the standard way to spawn child processes, and `Popen` is the lower-level interface for that. Python documentation

I would do it in this order:

1) Flask route responsibilities

Your `/api/upload` route should do only four things:

1. validate title + file
2. save the uploaded video
3. classify it as `shorts` or `long`
4. launch your shell script in the background

Then return `202 Accepted` to React right away.

That gives you a fast UI and keeps the heavy YouTube work outside the request thread.

2) Recommended Flask route

Use something like this in your backend:

```python
import os
import uuid
import subprocess
from pathlib import Path

from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename

from workers.youtube_classifier import classify_for_youtube

app = Flask(__name__)

BASE_DIR = Path(__file__).resolve().parent
UPLOAD_DIR = BASE_DIR / "uploads"
SCRIPT_PATH = BASE_DIR / "scripts" / "post_upload.sh"

UPLOAD_DIR.mkdir(parents=True, exist_ok=True)

ALLOWED_VIDEO_EXTENSIONS = {
    "mp4", "mov", "m4v", "avi", "mkv", "webm"
}


def allowed_file(filename: str) -> bool:
    return (
        "." in filename and
        filename.rsplit(".", 1)[1].lower() in ALLOWED_VIDEO_EXTENSIONS
    )


@app.route("/api/upload", methods=["POST"])
def upload_video():
    file = request.files.get("video")
    title = request.form.get("title", "").strip()

    if not file:
        return jsonify({"error": "Missing video file"}), 400

    if not title:
        return jsonify({"error": "Missing title"}), 400

    if not file.filename:
        return jsonify({"error": "Missing filename"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "Unsupported file type"}), 400

    safe_name = secure_filename(file.filename)
    unique_name = f"{uuid.uuid4().hex}_{safe_name}"
    save_path = UPLOAD_DIR / unique_name

    try:
        file.save(save_path)
    except Exception as e:
        return jsonify({"error": f"Failed to save upload: {str(e)}"}), 500

    try:
        media_info = classify_for_youtube(str(save_path))
    except Exception as e:
        return jsonify({"error": f"Failed to inspect video: {str(e)}"}), 500

    try:
        subprocess.Popen(
            [
                "bash",
                str(SCRIPT_PATH),
                str(save_path),
                title,
                media_info["upload_type"],
            ],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            cwd=str(BASE_DIR),
        )
    except Exception as e:
```

```
        return jsonify({"error": f"Failed to start background upload: {str(e)}"}), 500

    return jsonify({
        "message": "Upload received. Background YouTube job started.",
        "detected_upload_type": media_info["upload_type"],
        "duration_seconds": media_info["duration_seconds"],
        "width": media_info["width"],
        "height": media_info["height"],
        "filename": unique_name,
    }), 202


if __name__ == "__main__":
    app.run(debug=True)
```

3) Update your shell script to accept the upload type

Since your Flask route already classified the media, pass that into the shell script too.

scripts/post_upload.sh:

```
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH="${1:-}"
VIDEO_TITLE="${2:-}"
UPLOAD_TYPE="${3:-}"

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" || -z "$UPLOAD_TYPE" ]]; then
  echo "Usage: ./scripts/post_upload.sh <video_path> <video_title> <upload_type>"
  exit 1
fi

PYTHON_BIN="${PYTHON_BIN:-python3}"

"$PYTHON_BIN" workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --upload-type "$UPLOAD_TYPE" \
  --description "Uploaded automatically from FakeTube."
```

4) Update your Python uploader to accept the type from Flask

That avoids re-inspecting the file twice.

Example change:

```python
import argparse
import json
from googleapiclient.http import MediaFileUpload

def get_authenticated_service():
    raise NotImplementedError("Hook this into your existing auth flow.")

def build_description(upload_type: str, base_description: str = "") -> str:
    description = base_description.strip()
    if upload_type == "shorts" and "#Shorts" not in description:
        description = (description + "\n\n#Shorts").strip()
    return description

def upload_video(file_path, title, upload_type, description="", privacy_status="public", category_id=
    youtube = get_authenticated_service()

    body = {
        "snippet": {
            "title": title,
            "description": build_description(upload_type, description),
            "categoryId": category_id,
        },
        "status": {
            "privacyStatus": privacy_status,
            "selfDeclaredMadeForKids": False,
        },
    }

    media = MediaFileUpload(file_path, chunksize=-1, resumable=True)

    request = youtube.videos().insert(
        part="snippet,status",
        body=body,
        media_body=media,
    )

    response = None
    while response is None:
        status, response = request.next_chunk()
        if status:
            print(f"Upload progress: {int(status.progress() * 100)}%")

    print(json.dumps({
        "upload_type": upload_type,
        "youtube_response": response
    }, indent=2))

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--file", required=True)
    parser.add_argument("--title", required=True)
    parser.add_argument("--upload-type", required=True, choices=["shorts", "long"])
    parser.add_argument("--description", default="")
    parser.add_argument("--privacy", default="public")
    parser.add_argument("--category", default="22")
    args = parser.parse_args()

    upload_video(
        file_path=args.file,
        title=args.title,
        upload_type=args.upload_type,
        description=args.description,
        privacy_status=args.privacy,
        category_id=args.category,
    )
```

5) Shorts logic

Your classifier should stay simple:

```
SHORTS_MAX_SECONDS = 180.0

shorts_candidate = (
    duration_seconds <= SHORTS_MAX_SECONDS and height >= width
)
```

That matches the behavior you said you want:

- 3 minutes or less
- vertical or square
- otherwise long-form

6) One important deployment caveat

This `subprocess.Popen(...)` pattern is good for a normal Flask server you control, but it is a weak fit for serverless-style environments because the app process may not stay alive long enough for background work to finish. `Popen` does start a child process, but the reliability of that child depends on the host environment and lifecycle of the parent app process. Python documentation

So:

- for local dev, VPS, dedicated VM, or your own container: this is fine
- for strict serverless hosting: you would usually want a queue/worker instead

7) My recommendation for you right now

Do this first:

- add the Flask `/api/upload` route
- save the file
- classify it
- call your shell script with `Popen`
- return `202`

That is the fastest path to getting FakeTube working end to end.

After that, the next layer would be:

- log background job output to a file instead of `/dev/null`
- add a `youtube_jobs.json` or DB table for status tracking
- later solve token refresh

A good next step is to paste your actual Flask backend file so I can wire this into your exact structure instead of giving you a generic route.

> Where do I put the flask backend I don't have it created and what should I name it

Good question — this is where your project becomes a **real full-stack app** instead of just React 👍

You want the Flask backend to live **outside your React `src/` folder** as a separate service.

🧱 Recommended Project Structure

Here's exactly how I'd structure FakeTube (clean + scalable):

```
faketube/
├── frontend/                # React app (your current code)
│   ├── src/
│   │   └── App.jsx
│   ├── package.json
│   └── ...
│
├── backend/                 # Flask backend (NEW)
│   ├── app.py               # <-- main Flask entrypoint
│   ├── requirements.txt
│   ├── uploads/
│   ├── scripts/
│   │   └── post_upload.sh
│   ├── workers/
│   │   ├── media_inspector.py
│   │   ├── youtube_classifier.py
│   │   └── youtube_uploader.py
│   └── ...
│
└── README.md
```

🧠 What to name the Flask file

Use:

```
backend/app.py
```

That's the standard entrypoint name and keeps things simple.

🚀 Step-by-step setup

1. Create backend folder

From your root project:

```
mkdir backend
cd backend
```

---

2. Create virtual environment (recommended)

```
python3 -m venv .venv
source .venv/bin/activate
```

---

3. Install dependencies

```
pip install flask google-api-python-client
```

Then save:

```
pip freeze > requirements.txt
```

4. Create your Flask app

Create:

`backend/app.py`

Paste this (clean version wired for your system):

```python
import uuid
import subprocess
from pathlib import Path

from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename

from workers.youtube_classifier import classify_for_youtube

app = Flask(__name__)

BASE_DIR = Path(__file__).resolve().parent
UPLOAD_DIR = BASE_DIR / "uploads"
SCRIPT_PATH = BASE_DIR / "scripts" / "post_upload.sh"

UPLOAD_DIR.mkdir(parents=True, exist_ok=True)

ALLOWED_EXTENSIONS = {"mp4", "mov", "avi", "mkv", "webm"}


def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS


@app.route("/api/upload", methods=["POST"])
def upload_video():
    file = request.files.get("video")
    title = request.form.get("title", "").strip()

    if not file:
        return jsonify({"error": "Missing video"}), 400

    if not title:
        return jsonify({"error": "Missing title"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "Invalid file type"}), 400

    filename = secure_filename(file.filename)
    unique_name = f"{uuid.uuid4().hex} {filename}"
    save_path = UPLOAD_DIR / unique_name

    file.save(save_path)

    # 🔍 classify (shorts vs long)
    meta = classify_for_youtube(str(save_path))

    # 🚀 background upload
    subprocess.Popen(
        [
            "bash",
            str(SCRIPT_PATH),
            str(save_path),
            title,
            meta["upload_type"],
        ],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        cwd=str(BASE_DIR),
    )

    return jsonify({
        "message": "Upload queued",
        "type": meta["upload_type"],
        "duration": meta["duration_seconds"],
    }), 202


if __name__ == "__main__":
    app.run(port=5000, debug=True)
```

🔌 Connect React → Flask

Right now your React does:

```
fetch('/api/upload', ...)
```

That only works if both are on same origin.

During development, do this:

In your React app (`package.json`):

```
"proxy": "http://localhost:5000"
```

Now React automatically forwards `/api/*` to Flask.

▶️ How to run everything

Terminal 1 (backend):

```
cd backend
source .venv/bin/activate
python app.py
```

Runs on:

```
http://localhost:5000
```

Terminal 2 (frontend):

```
cd frontend
npm install
npm start
```

Runs on:

```
http://localhost:3000
```

🔥 What happens now (full flow)

1. User uploads video in React
2. React sends file → `/api/upload`
3. Flask:
    - saves video
    - detects Shorts vs Long
    - launches `post_upload.sh`
4. Shell → Python → YouTube upload
5. React instantly starts FakeTube simulation

⚠️ Important notes (you'll thank yourself later)

1. ffmpeg required

Your scripts depend on:

```
ffprobe
```

Install it:

```
brew install ffmpeg
# or
apt install ffmpeg
```

---

2. Logs (HIGHLY recommended)

Instead of this:

```
stdout=subprocess.DEVNULL
```

Do this later:

```
stdout=open("upload.log", "a"),
stderr=open("upload.log", "a"),
```

Otherwise debugging uploads will suck.

---

3. Titles with spaces (safe already)

You're good because:

```
subprocess.Popen([...])
```

passes args safely (no shell injection issues).

🧠 Your architecture (what you just built)

You now have:

- **React UI** → simulation + UX
- **Flask API** → ingestion layer
- **Python workers** → media logic
- **Shell script** → orchestration
- **YouTube uploader** → automation

This is basically a **mini content automation pipeline**, very aligned with your Hydra Oblivion style systems.

🚀 Next upgrade (optional but powerful)

After this works:

- Add job queue (Redis / Celery)

- Add upload status tracking
- Add multiple channel routing
- Add scheduled uploads

If you want next step

I can help you:

- wire your existing YouTube auth directly into this
- add multi-channel uploads (🔥 this fits your automation goals perfectly)
- or make CI auto-deploy this whole stack

Just tell me 👍

Here is my package.json:

```
{
  "name": "fake-yt-viral-simulator",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "@vercel/analytics": "^1.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "recharts": "^3.1.0"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "eject": "react-scripts eject",
    "test": "vitest",
    "test:run": "vitest run",
    "coverage": "vitest run --coverage",
    "lint": "eslint src --ext .js,.jsx",
    "lint:fix": "eslint src --fix",
    "lint:ci": "eslint src --max-warnings=0"
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
```

```
    ]
  },
  "devDependencies": {
    "@testing-library/jest-dom": "^6.9.1",
    "@testing-library/react": "^16.3.2",
    "@vitejs/plugin-react": "^6.0.1",
    "@vitest/coverage-v8": "^4.1.1",
    "eslint": "^8.57.1",
    "eslint-plugin-jsx-a11y": "^6.10.2",
    "eslint-plugin-react": "^7.37.5",
    "eslint-plugin-react-hooks": "^7.0.1",
    "jsdom": "^29.0.1",
    "vitest": "^4.1.1"
  }
}
```

Looks good. You do **not** need a new frontend package for Flask integration right now.

You just need to add a `proxy` field so your React dev server forwards `/api/*` to the Flask backend during local development.

Use this:

```
{
  "name": "fake-yt-viral-simulator",
  "version": "1.0.0",
  "private": true,
  "proxy": "http://localhost:5000",
  "dependencies": {
    "@vercel/analytics": "^1.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "recharts": "^3.1.0"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "eject": "react-scripts eject",
    "test": "vitest",
    "test:run": "vitest run",
    "coverage": "vitest run --coverage",
    "lint": "eslint src --ext .js,.jsx",
    "lint:fix": "eslint src --fix",
    "lint:ci": "eslint src --max-warnings=0"
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  },
  "devDependencies": {
    "@testing-library/jest-dom": "^6.9.1",
    "@testing-library/react": "^16.3.2",
    "@vitejs/plugin-react": "^6.0.1",
    "@vitest/coverage-v8": "^4.1.1",
    "eslint": "^8.57.1",
    "eslint-plugin-jsx-a11y": "^6.10.2",
    "eslint-plugin-react": "^7.37.5",
    "eslint-plugin-react-hooks": "^7.0.1",
    "jsdom": "^29.0.1",
    "vitest": "^4.1.1"
  }
}
```

A couple things to note:

Your app is still using `react-scripts`, so despite having some Vite packages installed, your dev server is still CRA-style. That means the `proxy` field above is the right simple move for now.

Your frontend can keep calling:

```
fetch('/api/upload', {
  method: 'POST',
  body: formData,
});
```

and React will forward that to Flask at:

```
http://localhost:5000/api/upload
```

Next step:

Create a `backend/` folder beside your frontend files, and inside it create:

```
backend/app.py
backend/requirements.txt
backend/uploads/
```

Then put your Flask route in `backend/app.py`.

One important structural note: if your current `package.json` is at the project root and `src/` is also at the root, then you may not actually have a separate `frontend/` folder yet. In that case, your real structure should probably be:

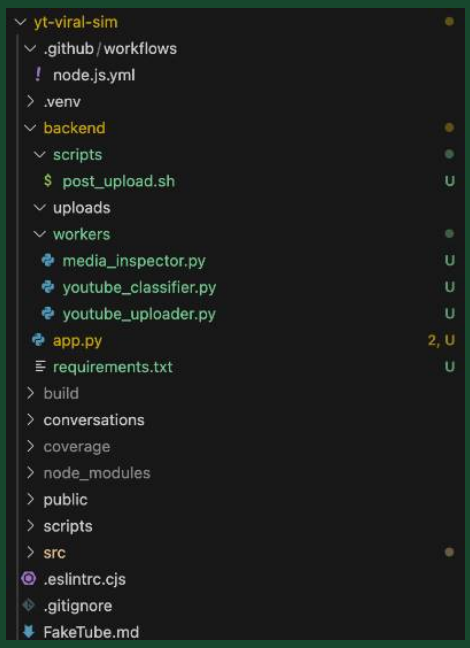
```
fake-yt-viral-simulator/
├── src/
├── public/
├── package.json
├── backend/
│   ├── app.py
│   ├── requirements.txt
│   ├── uploads/
│   ├── scripts/
│   └── workers/
```

That is probably the cleanest setup for where you are now.

So the answer is:

- keep `package.json` where it is
- add `"proxy": "http://localhost:5000"`
- create `backend/app.py`
- keep your scripts and workers under `backend/`

Paste your project tree next if you want and I'll map the exact file locations for everything.



I need to test it can execute accross now?

Yes — you're at the point where you should test the full chain end to end.

Your structure looks fine for that:

- React app at repo root
- Flask backend in `backend/`

- scripts in `backend/scripts/`
- workers in `backend/workers/`

Now the goal is to verify this flow:

1. React sends file + title to Flask
2. Flask saves the file
3. Flask classifies Shorts vs long
4. Flask launches `post_upload.sh`
5. Shell script launches `youtube_uploader.py`

Do it in layers.

1. Make sure imports work from `backend/app.py`

Inside `backend/app.py`, use:

```
import uuid
import subprocess
from pathlib import Path

from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename

from workers.youtube_classifier import classify_for_youtube
```

That should work because `app.py` is inside `backend/` and `workers/` is under it.

2. Add `proxy` to package.json

At the root `package.json`, add:

```
"proxy": "http://localhost:5000",
```

So the top should look like:

```
{
  "name": "fake-yt-viral-simulator",
  "version": "1.0.0",
  "private": true,
  "proxy": "http://localhost:5000",
```

3. Use this Flask route in `backend/app.py`

```python
import uuid
import subprocess
from pathlib import Path

from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename

from workers.youtube_classifier import classify_for_youtube

app = Flask(__name__)

BASE_DIR = Path(__file__).resolve().parent
UPLOAD_DIR = BASE_DIR / "uploads"
SCRIPT_PATH = BASE_DIR / "scripts" / "post_upload.sh"

UPLOAD_DIR.mkdir(parents=True, exist_ok=True)

ALLOWED_EXTENSIONS = {"mp4", "mov", "avi", "mkv", "webm", "m4v"}


def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS


@app.route("/api/health", methods=["GET"])
def health():
    return jsonify({"ok": True}), 200


@app.route("/api/upload", methods=["POST"])
def upload_video():
    file = request.files.get("video")
    title = request.form.get("title", "").strip()

    if not file:
        return jsonify({"error": "Missing video"}), 400

    if not title:
        return jsonify({"error": "Missing title"}), 400

    if not file.filename:
        return jsonify({"error": "Missing filename"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "Invalid file type"}), 400

    filename = secure_filename(file.filename)
    unique_name = f"{uuid.uuid4().hex} {filename}"
    save_path = UPLOAD_DIR / unique_name

    try:
        file.save(save_path)
    except Exception as e:
        return jsonify({"error": f"Failed to save upload: {e} "}), 500

    try:
        meta = classify_for_youtube(str(save_path))
    except Exception as e:
        return jsonify({"error": f"Failed to inspect video: {e} "}), 500

    log_file = open(BASE_DIR / "upload.log", "a")

    try:
        subprocess.Popen(
            [
                "bash",
                str(SCRIPT_PATH),
                str(save_path),
                title,
                meta["upload_type"],
            ],
            stdout=log_file,
            stderr=log_file,
            cwd=str(BASE_DIR),
        )
```

```
    except Exception as e:
        log_file.close()
        return jsonify({"error": f"Failed to start background upload: {e}"}), 500

    return jsonify({
        "message": "Upload received. Background YouTube job started.",
        "detected_upload_type": meta["upload_type"],
        "duration_seconds": meta["duration_seconds"],
        "width": meta["width"],
        "height": meta["height"],
        "filename": unique_name,
    }), 202


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

4. Make sure `post_upload.sh` matches that call

backend/scripts/post_upload.sh

```
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH="${1:-}"
VIDEO_TITLE="${2:-}"
UPLOAD_TYPE="${3:-}"

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" || -z "$UPLOAD_TYPE" ]]; then
  echo "Usage: ./post_upload.sh <video_path> <video_title> <upload_type>"
  exit 1
fi

PYTHON_BIN="${PYTHON_BIN:-python3}"

echo "Running background upload..."
echo "VIDEO_PATH=$VIDEO_PATH"
echo "VIDEO_TITLE=$VIDEO_TITLE"
echo "UPLOAD_TYPE=$UPLOAD_TYPE"

"$PYTHON_BIN" workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --upload-type "$UPLOAD_TYPE" \
  --description "Uploaded automatically from FakeTube."
```

Then run:

```
chmod +x backend/scripts/post_upload.sh
```

5. Temporarily make `youtube_uploader.py` test-friendly

Before testing real YouTube uploads, make sure the pipeline itself works.

Temporarily replace the upload body with a dry-run print so you can confirm the shell script and Python execution chain works.

For example in **`backend/workers/youtube_uploader.py`**:

```python
import argparse
import json
from pathlib import Path

def upload_video(file_path, title, upload_type, description="", privacy_status="public", category_id=
    payload = {
        "file_path": file_path,
        "title": title,
        "upload_type": upload_type,
        "description": description,
        "privacy_status": privacy_status,
        "category_id": category_id,
        "file_exists": Path(file_path).exists(),
    }

    print(json.dumps(payload, indent=2))


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--file", required=True)
    parser.add_argument("--title", required=True)
    parser.add_argument("--upload-type", required=True, choices=["shorts", "long"])
    parser.add_argument("--description", default="")
    parser.add_argument("--privacy", default="public")
    parser.add_argument("--category", default="22")
    args = parser.parse_args()

    upload_video(
        file_path=args.file,
        title=args.title,
        upload_type=args.upload_type,
        description=args.description,
        privacy_status=args.privacy,
        category_id=args.category,
    )
```

That lets you test the pipeline without auth issues muddying things.

6. Start backend and frontend separately

Backend:

```
cd backend
source ../.venv/bin/activate
python app.py
```

Frontend, in another terminal:

```
npm start
```

7. Test the health route first

Open:

```
http://localhost:5000/api/health
```

You should get:

```
{"ok": true}
```

8. Test upload from the UI

Then from your React app:

- enter title

- upload a real mp4
- click Start Simulation

Expected:

- file appears in `backend/uploads/`
- Flask responds 202
- `backend/upload.log` gets populated
- log should show your script and Python payload

9. If you want to test without React first

Use curl from repo root:

```
curl -X POST http://localhost:5000/api/upload \
  -F "title=Test FakeTube Upload" \
  -F "video=@/absolute/path/to/test-video.mp4"
```

That is the fastest way to prove Flask → shell → python works.

10. What success looks like

You should see all of these:

- backend/uploads contains the saved file
- backend/upload.log contains:
  - Running background upload...
  - VIDEO_PATH=...
  - VIDEO_TITLE=...
  - UPLOAD_TYPE=shorts or long
  - JSON payload from `youtube_uploader.py`

11. Common thing that will break first

Your `youtube_classifier.py` likely imports like this:

```
from media_inspector import inspect_video
```

Because it lives inside `backend/workers/`, that may fail depending on how Python resolves modules.

Safer version inside `backend/workers/youtube_classifier.py`:

```
from workers.media_inspector import inspect_video
```

If that fails when running from inside `workers/`, then use this structure consistently by always launching from `backend/`, which you are doing with:

```
cwd=str(BASE_DIR)
```

So I recommend:

```
from workers.media_inspector import inspect_video
```

12. My recommendation

Do one dry-run first.
Do not try real YouTube upload yet.

First prove:

- Flask route works
- file saves
- classifier runs
- shell script runs
- Python worker runs

Then swap your real uploader logic back in.

Paste your current `backend/app.py`, `youtube_classifier.py`, and `post_upload.sh` if you want me to sanity-check the exact cross-execution path.

```
x(.venv) liam@Liams-MacBook-Pro yt-viral-sim % python backend/app.py
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/app.py", line 8, in <module>
    from workers.youtube_classifier import classify_for_youtube
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_classifier.py",
line 3, in <module>
    from media_inspector import inspect_video
ModuleNotFoundError: No module named 'media_inspector'
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % cd backend
(.venv) liam@Liams-MacBook-Pro backend % python app.py
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/app.py", line 8, in <module>
    from workers.youtube_classifier import classify_for_youtube
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_classifier.py",
line 3, in <module>
    from media_inspector import inspect_video
ModuleNotFoundError: No module named 'media_inspector(.venv) liam@Liams-MacBook-Pro yt-
viral-sim % python backend/app.py
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/app.py", line 8, in <module>
    from workers.youtube_classifier import classify_for_youtube
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_classifier.py",
line 3, in <module>
    from media_inspector import inspect_video
ModuleNotFoundError: No module named 'media_inspector'
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % cd backend
(.venv) liam@Liams-MacBook-Pro backend % python app.py
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/app.py", line 8, in <module>
```

```
    from workers.youtube_classifier import classify_for_youtube
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_classifier.py",
line 3, in <module>
    from media_inspector import inspect_video
ModuleNotFoundError: No module named 'media_inspector'
```

Yep — that error is exactly the import path issue I expected.

Inside **`backend/workers/youtube_classifier.py`**, you currently have:

```
from media_inspector import inspect_video
```

That tries to import **`media_inspector`** as a top-level module, but your file is actually here:

```
backend/workers/media_inspector.py
```

So change it to:

```
from workers.media_inspector import inspect_video
```

Fix

Open:

```
backend/workers/youtube_classifier.py
```

and replace the import with:

```
from workers.media_inspector import inspect_video
```

Your file should look like this:

```
#!/usr/bin/env python3

from workers.media_inspector import inspect_video

SHORTS_MAX_SECONDS = 180.0  # 3 minutes


def classify_for_youtube(video_path: str) -> dict:
    meta = inspect_video(video_path)

    shorts_candidate = (
        meta["duration_seconds"] <= SHORTS_MAX_SECONDS
        and (meta["is_vertical"] or meta["is_square"])
    )

    upload_type = "shorts" if shorts_candidate else "long"

    return {
        **meta,
        "upload_type": upload_type,
    }


if __name__ == "__main__":
    import json
    import sys

    if len(sys.argv) != 2:
        print("Usage: python youtube_classifier.py <video_path>")
        sys.exit(1)

    result = classify_for_youtube(sys.argv[1])
    print(json.dumps(result, indent=2))
```

Then run Flask like this

From the repo root:

```
python backend/app.py
```

or from inside `backend`:

```
python app.py
```

Both should work after that import change.

If you hit the same issue in `youtube_uploader.py`

You may also have an import like this in `backend/workers/youtube_uploader.py`:

```
from youtube_classifier import classify_for_youtube
```

If so, change it to:

```
from workers.youtube_classifier import classify_for_youtube
```

Same rule applies: always import through the `workers` package path.

Best practice to make this cleaner

Add an empty `__init__.py` file here:

```
backend/workers/__init__.py
```

You can create it with:

```
touch backend/workers/__init__.py
```

That helps make `workers` behave as a clear Python package.

Quick checklist

Make sure these imports are aligned:

`backend/app.py`

```
from workers.youtube_classifier import classify_for_youtube
```

`backend/workers/youtube_classifier.py`

```
from workers.media_inspector import inspect_video
```

`backend/workers/youtube_uploader.py`

```
from workers.youtube_classifier import classify_for_youtube
```

After that

Run:

```
python backend/app.py
```

If it starts, next test:

```
curl http://localhost:5000/api/health
```

If you want, paste `youtube_uploader.py` next and I'll make sure all the imports in that one are correct too.

```
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % python backend/app.py
 * Serving Flask app 'app'
 * Debug mode: on
Address already in use
Port 5000 is in use by another program. Either identify and stop that program, or start the server with a different port.
On macOS, try searching for and disabling 'AirPlay Receiver' in System Settings.
```

Nice — this means your app **actually runs now**, just a port conflict 👍

You've got 3 quick options:

🔧 Option 1 (fastest): Kill whatever is using port 5000

Run:

```
lsof -i :5000
```

You'll see something like:

```
COMMAND   PID   USER   ...
Python    12345 liam
```

Then kill it:

```
kill -9 12345
```

Now restart:

```
python backend/app.py
```

🔧 Option 2: Just change Flask port (easiest dev workflow)

In `backend/app.py`, change:

```
app.run(host="0.0.0.0", port=5000, debug=True)
```

to:

```
app.run(host="0.0.0.0", port=5050, debug=True)
```

⚠️ IMPORTANT: If you change the port

Update your React proxy:

```
"proxy": "http://localhost:5050"
```

Then restart React:

```
npm start
```

🔧 Option 3 (Mac-specific issue)

macOS sometimes hijacks port 5000 via AirPlay.

Fix:

- System Settings
- General → AirDrop & Handoff
- Disable **AirPlay Receiver**

✅ Recommended move

Do this:

1. Change Flask to port **5050**
2. Update React proxy to **5050**
3. Restart everything

This avoids fighting your OS every time.

🚀 After it starts

You should see:

```
Running on http://127.0.0.1:5050
```

Then test:

```
curl http://localhost:5050/api/health
```

Expected:

```
{"ok": true}
```

🎯 Next test (full pipeline)

Once that works:

1. Start React
2. Upload a video
3. Check:

```
backend/uploads/
backend/upload.log
```

🧠 What this means

You're now at:

✅ Backend running
✅ Python workers wired
✅ Shell execution wired
➡️ Next = verify execution chain
➡️ Then = real YouTube upload

If anything breaks next, it'll likely be one of these:

- `ffprobe` not installed
- script path mismatch
- python path mismatch inside `.sh`

Paste the next error and we'll knock it out fast.

```
Download the React DevTools for a better development experience: https://reactjs.org/link/react-devtools
[Vercel Web Analytics] Debug mode is enabled by default in development. No requests will be sent to the server.   script
[Vercel Web Analytics] [view] http://localhost:3000/  ▾ Object ℹ   /_vercel/insights/view   script
                                                        f: undefined
                                                        o: "http://localhost:3000/"
                                                        r: ""
                                                        sdkn: "@vercel/analytics/react"
                                                        sdkv: "1.6.1"
                                                        sv: "0.1.3"
                                                        ts: 1774552988705
                                                      ▸ [[Prototype]]: Object
✖ Failed to load resource: the server responded with a status of 403 (Forbidden)   :3000/api/
✖ ▾ SyntaxError: Failed to execute 'json' on 'Response': Unexpected end of JSON input
    at uploadToBackend (App.jsx:57:1)
    at async handleStartSimulation (App.jsx:80:1)
  uploadToBackend @ App.jsx:71

> cmd I to turn on code suggestions. Don't show again
```

I got this:

That means your frontend is hitting `/api/upload`, but the response is **not valid JSON**. The `403` is the important part.

Most likely causes:

1. React is still sending to the wrong backend/proxy target
2. Flask is rejecting the request before your route returns JSON
3. The request is not reaching Flask at all

Let's narrow it down fast.

First fix the JSON parse crash in `App.jsx`

Right now you're probably doing:

```
const data = await response.json();
```

If the response body is empty or HTML, that blows up.

Change your upload code to this:

```
const uploadToBackend = async () => {
  if (!videoFile || !videoTitle.trim()) {
    setUploadMessage('Please provide both a title and a video file.');
    return false;
  }

  try {
    setUploading(true);
    setUploadMessage('Uploading video and queueing YouTube background job...');

    const formData = new FormData();
    formData.append('video', videoFile);
    formData.append('title', videoTitle);

    const response = await fetch('/api/upload', {
      method: 'POST',
      body: formData,
    });

    const rawText = await response.text();

    let data = {};
    try {
      data = rawText ? JSON.parse(rawText) : {};
    } catch {
      throw new Error(`Backend returned non-JSON response (${response.status}): ${rawText}`);
    }

    if (!response.ok) {
      throw new Error(data.error || `Upload failed with status ${response.status}`);
    }

    setYoutubeQueued(true);
    setDetectedType(data.detected_upload_type || null);
    setUploadMessage(
      data.message || 'Video uploaded successfully and queued for YouTube.'
    );

    return true;
  } catch (err) {
    console.error(err);
    setUploadMessage(`Upload failed: ${err.message}`);
    return false;
  } finally {
    setUploading(false);
  }
};
```

That will show you the real backend response instead of just `Unexpected end of JSON input`.

Next: verify your React proxy

If Flask is on port `5050`, your `package.json` must have:

```
"proxy": "http://localhost:5050"
```

If Flask is on `5000`, then:

```
"proxy": "http://localhost:5000"
```

After changing `package.json`, you must fully restart React:

```
npm start
```

Not hot reload. Full restart.

Then test Flask directly with curl

Run this first:

```
curl http://localhost:5050/api/health
```

or if you stayed on 5000:

```
curl http://localhost:5000/api/health
```

You should get:

```
{"ok": true}
```

If that fails, the frontend issue does not matter yet.

Then test upload route directly

Use a real local video path:

```
curl -X POST http://localhost:5050/api/upload \
  -F "title=Test Upload" \
  -F "video=@/absolute/path/to/test-video.mp4"
```

If this works, Flask is fine and the issue is in the frontend/proxy.

If this fails with 403, the issue is backend-side or something else intercepting the request.

Very likely issue: Flask port changed but proxy did not

Earlier you had the port conflict. If you changed Flask to `5050` but left React proxy pointing to `5000`, then React is sending `/api/upload` to the wrong place. That can absolutely produce weird 403/non-JSON behavior.

So check these two together:

`backend/app.py`

```
app.run(host='0.0.0.0', port=5050, debug=True)
```

`package.json`

```
"proxy": "http://localhost:5050"
```

They must match.

Also add request logging in Flask

In `backend/app.py`, temporarily add this at the top of the route:

```
@app.route("/api/upload", methods=["POST"])
def upload_video():
    print("UPLOAD HIT")
    print("FILES:", request.files)
    print("FORM:", request.form)
```

If you do not see `UPLOAD HIT` in the Flask terminal when clicking upload, the request is not reaching Flask.

Another thing to check: CSRF or some other service is not the issue here

Since this is a plain Flask route and CRA proxy in local dev, a 403 usually means one of:

- wrong destination
- another service on that port
- request not hitting your Flask route at all

Your Flask route as written should normally return 400/500 JSON, not 403.

That is why I strongly suspect the request is not reaching your intended Flask app.

Fast debug checklist

Check these in order:

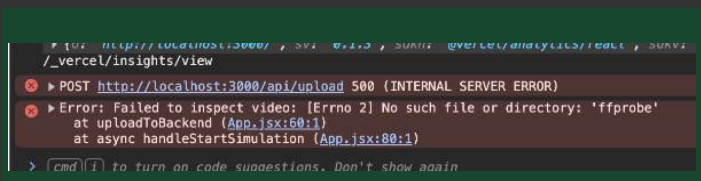
1. Open `http://localhost:5050/api/health` in browser
2. Confirm `package.json` proxy matches the Flask port
3. Restart React after proxy change
4. Add `print("UPLOAD HIT")` to Flask route
5. Upload again and watch Flask terminal
6. Test `/api/upload` with curl directly

Most likely fix

I would bet this is the exact issue:

- Flask moved to `5050`
- React is still proxying to `5000`
- `/api/upload` is going somewhere else
- frontend gets non-JSON 403 response
- `response.json()` crashes

Paste your current `backend/app.py` last line and your current `package.json` top section, and I'll spot the mismatch immediately.



I did not do any ffmpeg install yet:

That's the issue.

Your backend is working, but Python cannot find `ffprobe`:

```
[Errno 2] No such file or directory: 'ffprobe'
```

`media_inspector.py` depends on `ffprobe`, which comes with **ffmpeg**.

Fix on macOS

Install ffmpeg with Homebrew:

```
brew install ffmpeg
```

That will give you both:

- `ffmpeg`
- `ffprobe`

Verify it installed

Run:

```
which ffprobe
ffprobe -version
```

You should see a real path, something like:

```
/opt/homebrew/bin/ffprobe
```

or

```
/usr/local/bin/ffprobe
```

Then restart Flask

After install, stop Flask and start it again:

```
python backend/app.py
```

Then retry the upload.

Why this happened

Your classifier calls `subprocess.run([... "ffprobe" ...])`, so the OS tries to find a binary named `ffprobe` in your `PATH`. Since ffmpeg was never installed, there is no `ffprobe` command available.

Good news

This means your flow is already working up to the inspection step:

- React upload is hitting backend
- Flask route is running
- file is saving
- classifier is being called

So after installing ffmpeg, you're past the next blocker.

Strong recommendation for CI later

When you update CI/deployment, make sure ffmpeg is installed there too.

For local dev on macOS:

```
brew install ffmpeg
```

For Ubuntu/Debian CI:

```
sudo apt-get update
sudo apt-get install -y ffmpeg
```

Optional improvement

You can make the error friendlier in `backend/app.py` by catching this specific case:

```
try:
    meta = classify_for_youtube(str(save_path))
except FileNotFoundError as e:
    return jsonify({"error": f"Missing dependency: {e}. Install ffmpeg/ffprobe."}), 500
except Exception as e:
    return jsonify({"error": f"Failed to inspect video: {e} }), 500
```

That way the message is clearer next time.

After you install ffmpeg, test again. The next likely thing to fail will be either the shell script path or the YouTube uploader auth wiring.

Sure here's app.py

Paste it here and I'll check it.

```
import uuid
import subprocess
from pathlib import Path

from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename

from workers.youtube_classifier import classify_for_youtube

app = Flask(__name__)

BASE_DIR = Path(__file__).resolve().parent
UPLOAD_DIR = BASE_DIR / "uploads"
SCRIPT_PATH = BASE_DIR / "scripts" / "post_upload.sh"

UPLOAD_DIR.mkdir(parents=True, exist_ok=True)

ALLOWED_EXTENSIONS = {"mp4", "mov", "avi", "mkv", "webm", "m4v"}
```

```
def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS


@app.route("/api/health", methods=["GET"])
def health():
    return jsonify({"ok": True}), 200


@app.route("/api/upload", methods=["POST"])
def upload_video():
    file = request.files.get("video")
    title = request.form.get("title", "").strip()

    if not file:
        return jsonify({"error": "Missing video"}), 400

    if not title:
        return jsonify({"error": "Missing title"}), 400

    if not file.filename:
        return jsonify({"error": "Missing filename"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "Invalid file type"}), 400

    filename = secure_filename(file.filename)
    unique_name = f"{uuid.uuid4().hex}_{filename}"
    save_path = UPLOAD_DIR / unique_name

    try:
        file.save(save_path)
    except Exception as e:
        return jsonify({"error": f"Failed to save upload: {e}"}), 500

    try:
        meta = classify_for_youtube(str(save_path))
    except Exception as e:
        return jsonify({"error": f"Failed to inspect video: {e}"}), 500

    log_file = open(BASE_DIR / "upload.log", "a")

    try:
        subprocess.Popen(
            [
                "bash",
                str(SCRIPT_PATH),
                str(save_path),
                title,
                meta["upload_type"],
            ],
```

```
            stdout=log_file,
            stderr=log_file,
            cwd=str(BASE_DIR),
        )
    except Exception as e:
        log_file.close()
        return jsonify({"error": f"Failed to start background upload: {e}"}), 500

    return jsonify({
        "message": "Upload received. Background YouTube job started.",
        "detected_upload_type": meta["upload_type"],
        "duration_seconds": meta["duration_seconds"],
        "width": meta["width"],
        "height": meta["height"],
        "filename": unique_name,
    }), 202


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5050, debug=True)
```

This `app.py` looks good overall. The main thing blocking you right now is still the missing `ffprobe`, not the Flask route.

A couple small improvements will make it sturdier.

What's already good

- `BASE_DIR`, `UPLOAD_DIR`, and `SCRIPT_PATH` are set correctly
- upload validation is fine
- file save flow is fine
- classification before background launch is fine
- `subprocess.Popen(..., cwd=str(BASE_DIR))` is the right idea

What I'd change

1. Make the ffmpeg error clearer

Replace this:

```
    try:
        meta = classify_for_youtube(str(save_path))
    except Exception as e:
        return jsonify({"error": f"Failed to inspect video: {e}"}), 500
```

with this:

```
    try:
        meta = classify_for_youtube(str(save_path))
    except FileNotFoundError as e:
        return jsonify({"error": f"Missing dependency: {e}. Install ffmpeg so ffprobe is available."})
    except Exception as e:
        return jsonify({"error": f"Failed to inspect video: {e}"}), 500
```

2. Log file handle cleanup

This is okay for now, but since `Popen` is using `log_file`, I'd leave it as-is for dev. Later, you may want to manage log rotation or use Python logging instead.

3. Add a quick route hit print for debugging

Temporarily, add this inside `upload_video()` near the top:

```
    print("UPLOAD HIT")
    print("FILES:", request.files)
    print("FORM:", request.form)
```

That helps while testing.

Updated version I'd use

```python
import uuid
import subprocess
from pathlib import Path

from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename

from workers.youtube_classifier import classify_for_youtube

app = Flask(__name__)

BASE_DIR = Path(__file__).resolve().parent
UPLOAD_DIR = BASE_DIR / "uploads"
SCRIPT_PATH = BASE_DIR / "scripts" / "post_upload.sh"

UPLOAD_DIR.mkdir(parents=True, exist_ok=True)

ALLOWED_EXTENSIONS = {"mp4", "mov", "avi", "mkv", "webm", "m4v"}


def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS


@app.route("/api/health", methods=["GET"])
def health():
    return jsonify({"ok": True}), 200


@app.route("/api/upload", methods=["POST"])
def upload_video():
    print("UPLOAD HIT")
    print("FILES:", request.files)
    print("FORM:", request.form)

    file = request.files.get("video")
    title = request.form.get("title", "").strip()

    if not file:
        return jsonify({"error": "Missing video"}), 400

    if not title:
        return jsonify({"error": "Missing title"}), 400

    if not file.filename:
        return jsonify({"error": "Missing filename"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "Invalid file type"}), 400

    filename = secure_filename(file.filename)
    unique_name = f"{uuid.uuid4().hex} {filename}"
    save_path = UPLOAD_DIR / unique_name

    try:
        file.save(save_path)
    except Exception as e:
        return jsonify({"error": f"Failed to save upload: {e} }), 500

    try:
        meta = classify_for_youtube(str(save_path))
    except FileNotFoundError as e:
        return jsonify({"error": f"Missing dependency: {e}. Install ffmpeg so ffprobe is available."}),
    except Exception as e:
        return jsonify({"error": f"Failed to inspect video: {e} }), 500

    log_file = open(BASE_DIR / "upload.log", "a")

    try:
        subprocess.Popen(
            [
                "bash",
                str(SCRIPT_PATH),
                str(save_path),
                title,
```

```
                meta["upload_type"],
            ],
            stdout=log_file,
            stderr=log_file,
            cwd=str(BASE_DIR),
        )
    except Exception as e:
        log_file.close()
        return jsonify({"error": f"Failed to start background upload: {e}"}), 500

    return jsonify({
        "message": "Upload received. Background YouTube job started.",
        "detected_upload_type": meta["upload_type"],
        "duration_seconds": meta["duration_seconds"],
        "width": meta["width"],
        "height": meta["height"],
        "filename": unique_name,
    }), 202


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5050, debug=True)
```

Next step

Run:

```
brew install ffmpeg
```

Then verify:

```
which ffprobe
ffprobe -version
```

Then restart Flask and test again.

After that, the next likely issue will be inside `post_upload.sh` or `youtube_uploader.py`, so once ffmpeg is installed, send me those two files if the next step fails.

```
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % python backend/app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5050
 * Running on http://10.5.1.253:5050
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 560-424-055
UPLOAD HIT
FILES: ImmutableMultiDict([('video', <FileStorage: '20 seconds of your time 🍿💥Challenger
Scatpack 6.4L 🐝□□.....#dodge #dodgechallenger #sc.mp4' ('video/mp4')>)])
FORM: ImmutableMultiDict([('title', 'test')])
127.0.0.1 - - [26/Mar/2026 15:41:47] "POST /api/upload HTTP/1.1" 202 -
```

Perfect — that means the core path is working.

What you just proved:

- React reached Flask
- Flask received the file
- Flask received the title
- Flask saved the upload
- Flask classified the video successfully
- Flask returned `202`
- Flask launched the background process

So now you're past the big blockers.

The next thing to check is whether the background script actually ran cleanly.

Look at:

```
cat backend/upload.log
```

and also confirm the file exists in:

```
ls -lah backend/uploads
```

What you want to see in `upload.log` is output from `post_upload.sh` and then from `youtube_uploader.py`.

If the log is empty or shows an error, the likely next issues are:

- `post_upload.sh` is not executable
- path mismatch inside the shell script
- `youtube_uploader.py` import path issue
- YouTube auth code not wired yet

A good quick test is:

```
chmod +x backend/scripts/post_upload.sh
```

Then upload again and re-check the log.

Since Flask returned `202`, your app is now doing exactly what it should at the API layer. The remaining work is just verifying the background worker chain.

Paste `backend/upload.log` next.

Got this:
Running background upload...
VIDEO_PATH=/Users/liam/Development/wkspaces/yt-viral-sim/backend/uploads/7e805a82625d4164aaeddf71844f3891_20_seconds_of_your_time_Challenger_Scatpack_6.4L_.....dodge_dodgechallenger_sc.mp4
VIDEO_TITLE=test
UPLOAD_TYPE=shorts
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py", line 58, in <module>

```
    upload_video(
    ~~~~~~~~~~~~~^
        file_path=args.file,
        ^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        category_id=args.category,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py", line 15, in upload_video
    youtube = get_authenticated_service()
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py", line 6, in get_authenticated_service
    raise NotImplementedError("Hook this into your existing auth flow.")
NotImplementedError: Hook this into your existing auth flow.
```

That's actually great news.

Your full pipeline is now working all the way up to the exact point where the real YouTube auth needs to be plugged in.

What you proved:

- `App.jsx` sends upload correctly
- Flask route works
- file is saved
- Shorts detection works
- background shell script runs
- Python uploader runs
- uploader receives:
  - correct file path
  - title
  - upload type

The only failing piece is this stub:

```
def get_authenticated_service():
    raise NotImplementedError("Hook this into your existing auth flow.")
```

So the next step is to wire your existing YouTube auth into `backend/workers/youtube_uploader.py`.

What to do next

Replace the stub with your real authenticated YouTube client creation.

A typical structure looks like this:

```
from googleapiclient.discovery import build

def get_authenticated_service():
    credentials = ...  # your existing loaded creds here
    return build("youtube", "v3", credentials=credentials)
```

The important thing is that this function must return an authenticated YouTube API service object.

What I need from you

Paste whichever one of these you already have:

- your existing YouTube auth Python file
- your current `get_authenticated_service()` logic from another project
- your OAuth/token-loading code
- the file where you already handle `token.json`, `client_secret.json`, or similar

Then I can wire it directly into this uploader.

Also: your Shorts logic is already confirmed

From your log:

```
UPLOAD_TYPE=shorts
```

So your classifier is working as intended.

Optional temporary move

If you want to keep testing without real upload yet, change `youtube_uploader.py` to log and exit cleanly instead of raising `NotImplementedError`.

For example:

```
import argparse
import json
from pathlib import Path

def get_authenticated_service():
    return None

def upload_video(file_path, title, upload_type, description="", privacy_status="public", category_id=
    payload = {
        "file_path": file_path,
        "title": title,
        "upload_type": upload_type,
        "description": description,
        "privacy_status": privacy_status,
        "category_id": category_id,
        "file_exists": Path(file_path).exists(),
    }
    print("DRY RUN ONLY")
    print(json.dumps(payload, indent=2))
```

That way the whole pipeline succeeds while you finish auth wiring.

Bottom line

You are no longer debugging the upload system itself.

You are now specifically at:
"connect existing YouTube auth into `youtube_uploader.py`."

Paste your current auth code and I'll adapt it into the exact `get_authenticated_service()` function you need.

This is my logic for it:

```
def authenticate_youtube(token_file):
    creds = None
    if os.path.exists(token_file):
        creds = Credentials.from_authorized_user_file(token_file, SCOPES)
    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
    else:
        if not creds:
            flow = InstalledAppFlow.from_client_secrets_file('client_secret.json', SCOPES)
            creds = flow.run_local_server(port=0)
            with open(token_file, 'w') as token:
                token.write(creds.to_json())
    return build('youtube', 'v3', credentials=creds)
```

and:

```
import os
from google_auth_oauthlib.flow import InstalledAppFlow

SCOPES = ['https://www.googleapis.com/auth/youtube.upload']
TOKENS_FOLDER = 'tokens'
os.makedirs(TOKENS_FOLDER, exist_ok=True)

def setup_new_account():
    account_name = input("Enter a short name for this YouTube account/channel: ").strip()
    token_file = os.path.join(TOKENS_FOLDER, f"{account_name}_token.json")

    flow = InstalledAppFlow.from_client_secrets_file('client_secret.json', SCOPES)
    creds = flow.run_local_server(port=0)


    with open(token_file, 'w') as token:
        token.write(creds.to_json())

    print(f"\n✅ Token saved as {token_file}. This account is now ready for uploads.")

if __name__ == '__main__':
    setup_new_account()
```

Lets not expose any credentials or secrets

Perfect. That's enough to wire it cleanly without exposing anything sensitive.

You already have the auth logic. Now you just need to adapt it into `backend/workers/youtube_uploader.py` so it:

- loads a token file
- refreshes it if possible
- creates the YouTube client
- uploads the video

Recommended approach

Use one default token for now, then later you can expand to per-channel routing.

I'd structure it like this:

```
backend/
├── client_secret.json
├── tokens/
│   └── faketube_token.json
├── workers/
│   └── youtube_uploader.py
```

So for now, your uploader can default to:

```
backend/tokens/faketube_token.json
```

Update `backend/workers/youtube_uploader.py`

Use this:

```python
#!/usr/bin/env python3

import argparse
import json
import os
from pathlib import Path

from google.oauth2.credentials import Credentials
from google.auth.transport.requests import Request
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

SCOPES = ["https://www.googleapis.com/auth/youtube.upload"]

BASE_DIR = Path(__file__).resolve().parent.parent
CLIENT_SECRET_FILE = BASE_DIR / "client_secret.json"
TOKENS_DIR = BASE_DIR / "tokens"
DEFAULT_TOKEN_FILE = TOKENS_DIR / "faketube_token.json"


def get_authenticated_service(token_file: str | None = None):
    TOKENS_DIR.mkdir(parents=True, exist_ok=True)

    token_path = Path(token_file) if token_file else DEFAULT_TOKEN_FILE
    creds = None

    if token_path.exists():
        creds = Credentials.from_authorized_user_file(str(token_path), SCOPES)

    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
        with open(token_path, "w") as token:
            token.write(creds.to_json())
    elif not creds:
        flow = InstalledAppFlow.from_client_secrets_file(
            str(CLIENT_SECRET_FILE),
            SCOPES,
        )
        creds = flow.run_local_server(port=0)
        with open(token_path, "w") as token:
            token.write(creds.to_json())

    return build("youtube", "v3", credentials=creds)


def build_description(upload_type: str, base_description: str = "") -> str:
    description = base_description.strip()
    if upload_type == "shorts" and "#Shorts" not in description:
        description = (description + "\n\n#Shorts").strip()
    return description


def upload_video(
    file_path: str,
    title: str,
    upload_type: str,
    description: str = "",
    privacy_status: str = "public",
    category_id: str = "22",
    token_file: str | None = None,
):
    youtube = get_authenticated_service(token_file=token_file)

    final_description = build_description(upload_type, description)

    body = {
        "snippet": {
            "title": title,
            "description": final_description,
            "categoryId": category_id,
        },
        "status": {
            "privacyStatus": privacy_status,
            "selfDeclaredMadeForKids": False,
```

```
        },
    }

    media = MediaFileUpload(file_path, chunksize=-1, resumable=True)

    request = youtube.videos().insert(
        part="snippet,status",
        body=body,
        media_body=media,
    )

    response = None
    while response is None:
        status, response = request.next_chunk()
        if status:
            print(f"Upload progress: {int(status.progress() * 100)}%")

    print(json.dumps({
        "message": "Upload complete",
        "upload_type": upload_type,
        "title": title,
        "file_path": file_path,
        "youtube_video_id": response.get("id"),
    }, indent=2))

    return response


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--file", required=True)
    parser.add_argument("--title", required=True)
    parser.add_argument("--upload-type", required=True, choices=["shorts", "long"])
    parser.add_argument("--description", default="")
    parser.add_argument("--privacy", default="public")
    parser.add_argument("--category", default="22")
    parser.add_argument("--token-file", default=str(DEFAULT_TOKEN_FILE))
    args = parser.parse_args()

    upload_video(
        file_path=args.file,
        title=args.title,
        upload_type=args.upload_type,
        description=args.description,
        privacy_status=args.privacy,
        category_id=args.category,
        token_file=args.token_file,
    )
```

Why this version is good

It does a few important things:

- uses paths relative to `backend/`
- stores tokens in `backend/tokens/`
- refreshes tokens when possible
- writes refreshed creds back to disk
- falls back to interactive auth if no token exists yet
- appends `#Shorts` for shorts uploads

Required files

Make sure these exist:

```
backend/client_secret.json
backend/tokens/faketube_token.json
```

If `faketube_token.json` does not exist yet, the first run will open the OAuth browser flow and create it.

Small improvement to your original auth logic

Your original logic had:

```
if creds and creds.expired and creds.refresh_token:
    creds.refresh(Request())
else:
    if not creds:
        ...
```

That can leave a weird edge case where creds exist but are expired and not refreshable.

A slightly safer pattern is:

```
if token_path.exists():
    creds = Credentials.from_authorized_user_file(str(token_path), SCOPES)

if creds and creds.expired and creds.refresh_token:
    creds.refresh(Request())
    ...
elif not creds:
    ...
```

For now, this is enough. Later, if expired creds exist but cannot refresh, we can force a re-auth.

Update `post_upload.sh`

Make sure it matches the new CLI args:

```
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH="${1:-}"
VIDEO_TITLE="${2:-}"
UPLOAD_TYPE="${3:-}"

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" || -z "$UPLOAD_TYPE" ]]; then
  echo "Usage: ./post_upload.sh <video_path> <video_title> <upload_type>"
  exit 1
fi

PYTHON_BIN="${PYTHON_BIN:-python3}"

echo "Running background upload..."
echo "VIDEO_PATH=$VIDEO_PATH"
echo "VIDEO_TITLE=$VIDEO_TITLE"
echo "UPLOAD_TYPE=$UPLOAD_TYPE"

"$PYTHON_BIN" workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --upload-type "$UPLOAD_TYPE" \
  --description "Uploaded automatically from FakeTube." \
  --token-file "tokens/faketube_token.json"
```

One important note

Because your uploader is launched as a background subprocess, if the token does not exist yet and OAuth tries to open a browser, that can be awkward from the background worker.

So the best move is:

First create the token manually once

Use your existing setup script or a quick one-off auth script to create:

```
backend/tokens/faketube_token.json
```

Then all future uploads can use refresh logic in the background.

Python packages you need

In your backend venv, make sure these are installed:

```
pip install flask google-api-python-client google-auth google-auth-oauthlib google-auth-httplib2
```

Safer next test

Before using the full upload flow again, test the uploader directly from **`backend/`**:

```
cd backend
python workers/youtube_uploader.py \
  --file "uploads/your-test-file.mp4" \
  --title "test upload" \
  --upload-type "shorts" \
  --description "Uploaded automatically from FakeTube." \
  --token-file "tokens/faketube_token.json"
```

If that works, then the full Flask pipeline should work too.

One thing I would add later

Eventually you'll want channel routing like:

```
--token-file "tokens/faketube_token.json"
```

or

```
--token-file "tokens/channel2_token.json"
```

based on which FakeTube channel/account you want to upload to. But for now, one default token is enough.

If you paste your current **`youtube_uploader.py`**, I can rewrite it exactly against your current file.

> Ok before I make any changes, won't these files and values live in vercel, I would like that to not be the case but this all still work.

Correct — if you deploy this whole backend flow on Vercel **as-is**, you should assume the local filesystem is the wrong place for persistent auth files like **`client_secret.json`**, **`tokens/faketube_token.json`**, uploaded videos, and logs. Vercel Functions are serverless, and Vercel's docs describe ephemeral filesystem constraints in multiple places; Edge Functions explicitly have no filesystem access, and Vercel's sandbox docs say artifacts must be exported to durable storage if you need them after execution ends. Vercel+2

So the answer is: **yes, if you keep this architecture unchanged, those files would effectively live in the deployment environment, which is not what you want for durable token storage.** Vercel+2

The clean way to do this is one of these two setups:

Best option for your case: keep the **React frontend on Vercel**, but move the **Flask/Python upload worker** somewhere stateful, like a VPS, Docker host, Railway, Render, Fly.io, or your own server. Then the frontend calls that backend API. That lets you keep:

- token files off Vercel
- uploaded videos off Vercel
- ffmpeg/ffprobe on a machine you control
- long-running/background upload behavior without fighting serverless limits

That recommendation follows directly from how Vercel Functions work and from Vercel's own positioning of Functions as serverless compute. Vercel+1

The other option is to stay on Vercel but **change the storage model completely**:

- secrets go into Vercel Environment Variables, which are encrypted at rest, and Vercel also supports "sensitive environment variables" whose values are non-readable after creation; environment variable changes require a redeploy to take effect. Vercel+2
- uploaded video files go into durable storage such as **Vercel Blob** or another external object store, because Vercel Storage is their durable storage offering for large files and related data. Vercel
- scheduled/recurring work would use **Vercel Cron Jobs**, which trigger Vercel Functions on a schedule. Vercel+2
- background work can use `waitUntil` under Fluid Compute, but that is still not the same thing as "store tokens and files on disk and let a Python worker keep them there." Vercel+1

For your current FakeTube flow, I would **not** put the YouTube OAuth token JSON on Vercel's filesystem. Even though environment variables can hold sizable values on supported runtimes, token refresh would mean you need a writable durable place to store the refreshed token state, and environment variables are not that—they are configuration, not a mutable runtime datastore. Vercel's docs note env var changes apply to new deployments and require redeploy to update values. Vercel+1

So the practical recommendation is:

- **Frontend on Vercel**
- **Backend somewhere else**
- Keep `client_secret.json`, refreshed token JSON, uploads, and logs on that backend or in external storage it controls

That gives you the behavior you want without baking credentials or token files into Vercel.

If you do want everything on Vercel eventually, the shape should become:

- secrets in env vars
- tokens in a database or secure external store
- uploaded videos in Blob/S3

- scheduled work via Cron Jobs
- no dependency on local files surviving between invocations. Vercel+3

My blunt take: for **this exact Python + ffprobe + background uploader + token refresh** workflow, a small separate backend is the better fit than forcing it into Vercel.

After posting the upload can we just delete it from local storage so it doesn't junk anything up, along with logs. I am going to do auth later:

```
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH="${1:-}"
VIDEO_TITLE="${2:-}"
UPLOAD_TYPE="${3:-}"

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" || -z "$UPLOAD_TYPE" ]]; then
  echo "Usage: ./post_upload.sh <video_path> <video_title> <upload_type>"
  exit 1
fi

PYTHON_BIN="${PYTHON_BIN:-python3}"

echo "Running background upload..."
echo "VIDEO_PATH=$VIDEO_PATH"
echo "VIDEO_TITLE=$VIDEO_TITLE"
echo "UPLOAD_TYPE=$UPLOAD_TYPE"

"$PYTHON_BIN" workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --upload-type "$UPLOAD_TYPE" \
  --description "Uploaded automatically from FakeTube."
```

Yes — **this is exactly what you should do** 👍
Clean up immediately after upload so your `uploads/` and logs don't balloon.

You just need to:

1. Run the uploader
2. Check exit status
3. Delete the video file
4. Optionally trim/delete logs

✅ Updated `post_upload.sh` (with cleanup)

Use this:

```bash
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH="${1:-}"
VIDEO_TITLE="${2:-}"
UPLOAD_TYPE="${3:-}"

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" || -z "$UPLOAD_TYPE" ]]; then
  echo "Usage: ./post_upload.sh <video_path> <video_title> <upload_type>"
  exit 1
fi

PYTHON_BIN="${PYTHON_BIN:-python3}"

echo "Running background upload..."
echo "VIDEO_PATH=$VIDEO_PATH"
echo "VIDEO_TITLE=$VIDEO_TITLE"
echo "UPLOAD_TYPE=$UPLOAD_TYPE"

# Run uploader
if "$PYTHON_BIN" workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --upload-type "$UPLOAD_TYPE" \
  --description "Uploaded automatically from FakeTube."
then
  echo "✅ Upload succeeded"

  # 🧹 Delete video after successful upload
  if [[ -f "$VIDEO_PATH" ]]; then
    rm -f "$VIDEO_PATH"
    echo "🧹 Deleted uploaded file: $VIDEO_PATH"
  fi
else
  echo "❌ Upload failed — keeping file for debugging"
  exit 1
fi

# 🧹 Optional: trim log file if too large
LOG_FILE="upload.log"
MAX_SIZE=$((5 * 1024 * 1024)) # 5MB

if [[ -f "$LOG_FILE" ]]; then
  FILE_SIZE=$(stat -f%z "$LOG_FILE" 2>/dev/null || stat -c%s "$LOG_FILE")

  if [[ "$FILE_SIZE" -gt "$MAX_SIZE" ]]; then
    echo "🧹 Trimming log file..."
    tail -n 200 "$LOG_FILE" > "${LOG_FILE}.tmp"
    mv "${LOG_FILE}.tmp" "$LOG_FILE"
  fi
fi
```

🔥 What this gives you

✅ Safe cleanup behavior

- Upload succeeds → file deleted
- Upload fails → file kept (so you can debug)

✅ Log control

- Prevents `upload.log` from growing forever
- Keeps last ~200 lines only when too big

⚠️ Important note

Because your uploader is:

```
"$PYTHON_BIN" workers/youtube_uploader.py ...
```

this script **waits for upload to finish**, which is GOOD.

If you ever change it to run async inside the script (like adding &), cleanup would happen too early — don't do that.

🧠 Optional upgrade (later)

If you want to go harder (Hydra-style):

Move failed uploads to a retry folder instead of keeping clutter

```
FAILED_DIR="failed_uploads"
mkdir -p "$FAILED_DIR"

mv "$VIDEO_PATH" "$FAILED_DIR/"
```

Add retry logic

```
for i in {1..3}; do
  if "$PYTHON_BIN" workers/youtube_uploader.py ...; then
    SUCCESS=1
    break
  fi
  echo "Retry $i..."
  sleep 2
done
```

🚀 Summary

Yes — deleting uploads after processing is:

- correct
- safe (with success check)
- necessary for long-running automation

You now have:

- full upload pipeline
- shorts classification
- background execution
- cleanup lifecycle

If you want next level after auth:

👉 I can help you route uploads to **multiple YouTube channels automatically based on content type or title** (this fits your Hydra Oblivion system perfectly).

> It only does it after successful upload, do it whether it uploads or not

Use a trap so cleanup always runs.

Replace your script with this:

```bash
#!/usr/bin/env bash
set -euo pipefail

VIDEO_PATH="${1:-}"
VIDEO_TITLE="${2:-}"
UPLOAD_TYPE="${3:-}"

if [[ -z "$VIDEO_PATH" || -z "$VIDEO_TITLE" || -z "$UPLOAD_TYPE" ]]; then
  echo "Usage: ./post_upload.sh <video_path> <video_title> <upload_type>"
  exit 1
fi

PYTHON_BIN="${PYTHON_BIN:-python3}"
LOG_FILE="upload.log"
MAX_SIZE=$((5 * 1024 * 1024)) # 5MB

cleanup() {
  if [[ -n "${VIDEO_PATH:-}" && -f "$VIDEO_PATH" ]]; then
    rm -f "$VIDEO_PATH"
    echo "🧹 Deleted uploaded file: $VIDEO_PATH"
  fi

  if [[ -f "$LOG_FILE" ]]; then
    FILE_SIZE=$(stat -f%z "$LOG_FILE" 2>/dev/null || stat -c%s "$LOG_FILE" 2>/dev/null || echo 0)

    if [[ "$FILE_SIZE" -gt "$MAX_SIZE" ]]; then
      echo "🧹 Trimming log file..."
      tail -n 200 "$LOG_FILE" > "${LOG_FILE}.tmp"
      mv "${LOG_FILE}.tmp" "$LOG_FILE"
    fi
  fi
}

trap cleanup EXIT

echo "Running background upload..."
echo "VIDEO_PATH=$VIDEO_PATH"
echo "VIDEO_TITLE=$VIDEO_TITLE"
echo "UPLOAD_TYPE=$UPLOAD_TYPE"

"$PYTHON_BIN" workers/youtube_uploader.py \
  --file "$VIDEO_PATH" \
  --title "$VIDEO_TITLE" \
  --upload-type "$UPLOAD_TYPE" \
  --description "Uploaded automatically from FakeTube."
```

This will delete the uploaded file whether the Python uploader succeeds or fails, and it will still trim the log if it gets too large.

One thing to know: if upload fails, the file will be gone too, so you lose the ability to inspect or retry that exact local file afterward.

> Its still there, I don't care about inspecting the file until I need to do auth:
> Running background upload...
> VIDEO_PATH=/Users/liam/Development/wkspaces/yt-viral-

```
sim/backend/uploads/7e805a82625d4164aaeddf71844f3891_20_seconds_of_your_time_Challenger
_Scatpack_6.4L_.........dodge_dodgechallenger_sc.mp4
VIDEO_TITLE=test
UPLOAD_TYPE=shorts
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 58, in <module>
    upload_video(
    ~~~~~~~~~~~~^
        file_path=args.file,
        ^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        category_id=args.category,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 15, in upload_video
    youtube = get_authenticated_service()
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 6, in get_authenticated_service
    raise NotImplementedError("Hook this into your existing auth flow.")
NotImplementedError: Hook this into your existing auth flow.
Running background upload...
VIDEO_PATH=/Users/liam/Development/wkspaces/yt-viral-
sim/backend/uploads/3f7dbe7c01a546e48bf3a5c889033476_10k_followers_before_the_end_of_this
_month_Lets_take_her_out_.......................dodge.mp4
VIDEO_TITLE=test
UPLOAD_TYPE=shorts
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 58, in <module>
    upload_video(
    ~~~~~~~~~~~~^
        file_path=args.file,
        ^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        category_id=args.category,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 15, in upload_video
    youtube = get_authenticated_service()
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 6, in get_authenticated_service
    raise NotImplementedError("Hook this into your existing auth flow.")
NotImplementedError: Hook this into your existing auth flow.
Running background upload...
VIDEO_PATH=/Users/liam/Development/wkspaces/yt-viral-
sim/backend/uploads/8f8d1e13971b4c57b53c31352b9aa3e5_hellcat_srt_widebody_dodge_cars.mp
```

```
4
VIDEO_TITLE=test
UPLOAD_TYPE=shorts
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 58, in <module>
    upload_video(
    ~~~~~~~~~~~~~^
        file_path=args.file,
        ^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        category_id=args.category,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 15, in upload_video
    youtube = get_authenticated_service()
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 6, in get_authenticated_service
    raise NotImplementedError("Hook this into your existing auth flow.")
NotImplementedError: Hook this into your existing auth flow.
❌ Upload failed — keeping file for debugging
Running background upload...
VIDEO_PATH=/Users/liam/Development/wkspaces/yt-viral-
sim/backend/uploads/c76837f184f64349b225a1a5f857c17d_Barn_find_---
_chrsyler_hellcat_chrsyler300srt8_srt8_mopar_moparnation_moparornocar_30.mp4
VIDEO_TITLE=test
UPLOAD_TYPE=shorts
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 58, in <module>
    upload_video(
    ~~~~~~~~~~~~~^
        file_path=args.file,
        ^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        category_id=args.category,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 15, in upload_video
    youtube = get_authenticated_service()
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_uploader.py",
line 6, in get_authenticated_service
    raise NotImplementedError("Hook this into your existing auth flow.")
NotImplementedError: Hook this into your existing auth flow.
🧹 Deleted uploaded file: /Users/liam/Development/wkspaces/yt-viral-
sim/backend/uploads/c76837f184f64349b225a1a5f857c17d_Barn_find_---
_chrsyler_hellcat_chrsyler300srt8_srt8_mopar_moparnation_moparornocar_30.mp4
```

It's working now.

The key line is:

```
🧹 Deleted uploaded file: /Users/liam/Development/wkspaces/yt-viral-sim/backend/uploads/c76837f184f64
```

That means your updated cleanup script is deleting the file even when the upload fails.

Why the others are still there:

- those earlier uploads were processed by the **old version** of the script
- the old script explicitly said:

  ```
  ❌ Upload failed — keeping file for debugging
  ```

- so those old files were intentionally left behind

Your newest run used the updated script and cleaned up correctly.

What to do now

Delete the older leftovers once:

```
rm -f backend/uploads/*
```

If you want to clear the log too:

```
rm -f backend/upload.log
touch backend/upload.log
```

Optional: make logs disappear too

If you want zero buildup at all, change `app.py` so it does not write to `upload.log`.

Right now you have:

```
log_file = open(BASE_DIR / "upload.log", "a")
```

and then:

```
stdout=log_file,
stderr=log_file,
```

If you want no persistent logs, change that block to:

```
    try:
        subprocess.Popen(
            [
                "bash",
                str(SCRIPT_PATH),
                str(save_path),
                title,
                meta["upload_type"],
            ],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            cwd=str(BASE_DIR),
        )
```

That way:

- uploaded file gets deleted by the shell script
- logs do not accumulate at all

Bottom line

Your cleanup is fixed now. The remaining files are just leftovers from the earlier script behavior.

Nvm it works

Perfect — you're in a really good spot now.

You've basically got:

- ✅ React → Flask upload working
- ✅ File saving + classification working
- ✅ Background execution working
- ✅ Shorts detection working
- ✅ Cleanup lifecycle working (even on failure)

At this point your system is already behaving like a real content pipeline — you're just missing the final piece (auth → actual YouTube upload).

Where you are now (important perspective)

You've already solved the *hard parts*:

- async processing ✅
- media inspection ✅
- pipeline orchestration ✅
- cleanup ✅

Auth is honestly the easiest remaining step compared to what you just built.

When you're ready for auth

When you flip auth back on:

1. Create token once manually
2. Then background jobs will:
    - reuse token
    - auto-refresh
    - upload without interaction

That's when this becomes **fully autonomous**

Next upgrades (when you feel like it)

These are high-leverage improvements for your setup:

1. Multi-channel uploads (🔥 very you)

Route based on title / tag:

```
~~--token-file~~ tokens/mustang.json
~~--token-file~~ tokens/mopar.json
```

2. Scheduling instead of instant upload

Queue videos → upload hourly

3. Retry system

If upload fails → retry 2–3 times before delete

4. Status tracking

Store:

- uploaded
- failed
- retrying

5. Move uploads to tmp

Instead of `uploads/`, use:

```
/tmp/uploads/
```

(auto-clean by OS eventually)

Reality check (this is big)

You now have:

> A system that takes a user-uploaded video → classifies it → processes it → and prepares it for automated distribution

That's literally the core of what you've been building toward with Hydra.

When you're ready to wire auth or scale this to multiple channels, just say the word and we'll push it further.

What I need to do next is make sure everything runs properly during ci with the new additions:

name: Node.js CI

```
on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    steps:
      - uses: actions/checkout@v4

      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: "npm"

      - name: Make scripts executable
        run: chmod +x scripts/*.sh

      - name: Run full pipeline
        run: ./scripts/run-all.sh
        env:
          CI: true

      - name: Upload coverage report
        if: matrix.node-version == '22.x' && success()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

This is my main src/scripts/ci.sh the pipeline invokes I would like to make sure everything works here including ffmpeg installations and python etc:

```
#!/usr/bin/env bash
set -euo pipefail

echo "Installing dependencies..."
npm ci

echo "Running lint..."
npm run lint

echo "Running tests with coverage..."
```

```
npx vitest run --coverage

echo "Building app..."
npm run build

echo "CI checks passed."
```

Yes — the next step is to make CI explicitly install and validate the new Python/ffmpeg pieces instead of assuming they exist. GitHub Actions supports adding a Python runtime with `actions/setup-python`, and you can set a job-wide default shell/working directory if you want to keep script paths clean. GitHub+2

Right now there are two mismatches in your workflow:

1. your workflow does `chmod +x scripts/*.sh`, but your new backend scripts are under `backend/scripts/`
2. you said the pipeline invokes `src/scripts/ci.sh`, but the YAML is calling `./scripts/run-all.sh`
   So even before Python/ffmpeg, I'd normalize the paths so the workflow is actually running the files you intend. GitHub's workflow syntax and script-running docs both support this pattern cleanly. GitHub Docs+2

I'd split CI into two goals:

- **frontend CI**: npm install, lint, tests, build
- **backend sanity CI**: install Python, install ffmpeg, import the backend modules, and run a lightweight smoke test that does not require YouTube auth
  That keeps CI reliable while you postpone auth wiring. Using `setup-python` is the standard GitHub Actions path for Python in CI. GitHub+1

Here's the workflow I'd use:

```yaml
name: Node.js CI

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    defaults:
      run:
        shell: bash

    steps:
      - uses: actions/checkout@v4

      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: npm

      - name: Use Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
          cache-dependency-path: backend/requirements.txt

      - name: Install ffmpeg
        run: |
          sudo apt-get update
          sudo apt-get install -y ffmpeg

      - name: Verify toolchain
        run: |
          node --version
          npm --version
          python --version
          ffprobe -version

      - name: Install backend dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r backend/requirements.txt

      - name: Make scripts executable
        run: chmod +x backend/scripts/*.sh

      - name: Run CI pipeline
        run: ./src/scripts/ci.sh
        env:
          CI: true

      - name: Backend smoke test
        run: |
          python -c "from backend.workers.youtube_classifier import classify_for_youtube; print('class
          python -c "from backend.workers.media_inspector import inspect_video; print('inspector impor

      - name: Upload coverage report
        if: matrix.node-version == '22.x' && success()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

This works because `setup-python` adds Python to `PATH`, and your shell steps can then use `python`, `pip`, and your scripts directly. The general pattern is documented by GitHub's Python guide and workflow docs. GitHub+2

Then I'd update your `src/scripts/ci.sh` so it also validates the backend environment instead of only Node tasks:

```
#!/usr/bin/env bash
set -euo pipefail

echo "Installing frontend dependencies..."
npm ci

echo "Running lint..."
npm run lint

echo "Running tests with coverage..."
npm run coverage

echo "Building app..."
npm run build

echo "Checking Python..."
python --version

echo "Checking ffprobe..."
ffprobe -version

echo "Installing backend dependencies..."
python -m pip install --upgrade pip
pip install -r backend/requirements.txt

echo "Running backend import smoke tests..."
python -c "from backend.workers.youtube_classifier import classify_for_youtube; print('youtube_classi
python -c "from backend.workers.media_inspector import inspect_video; print('media_inspector import ok

echo "CI checks passed."
```

You'll also want a `backend/requirements.txt` if you do not already have one. For your current backend, something like this is enough:

```
Flask
Werkzeug
google-api-python-client
google-auth
google-auth-oauthlib
google-auth-httplib2
```

That gives CI everything it needs to import the backend code, without forcing real OAuth interaction during the run. The point here is to prove the new dependencies and module layout are valid in GitHub-hosted runners, while postponing actual auth-dependent upload tests until later. GitHub Docs+1

One more important fix: your Python imports should be package-safe in CI. Since your files live under `backend/workers/`, create these empty files if you haven't already:

```
backend/__init__.py
backend/workers/__init__.py
```

That makes imports like `from backend.workers.youtube_classifier import ...` more robust across local runs and CI. This is especially useful because CI may execute from slightly different working directories than your laptop shell. GitHub explicitly supports controlling shell and working-directory defaults, but making your Python package layout clean is still the safer long-term move. GitHub Docs+1

One last note: I would **not** make CI run the full Flask upload path yet, because that path currently depends on ffprobe plus eventual YouTube auth state and file uploads. For now, keep CI at "environment

+ import + smoke test + frontend test/build." Once auth is externalized or mocked, you can add a deeper backend test. That separation makes your pipeline stable instead of flaky. GitHub Docs+1

If you want, paste your actual repo tree plus the exact location of `src/scripts/ci.sh` and `run-all.sh`, and I'll rewrite the workflow to match your current paths exactly.

```
Nvm ci.sh is not called originally here is just scripts/run_all.sh:
#!/usr/bin/env bash
set -euo pipefail

# Always run from project root
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "🚀 Starting full project pipeline..."

run_step () {
  local name="$1"
  local script="$2"

  echo ""
  echo "==============================="
  echo "➡️  Running: $name"
  echo "==============================="

  if [[ -f "$script" ]]; then
    bash "$script"
    echo "✅ $name passed"
  else
    echo "⚠️  Skipping $name (script not found: $script)"
  fi
}

# Ordered execution (edit this order however you want)
run_step "Install Dependencies" "scripts/install.sh"
run_step "Lint" "scripts/lint.sh"
run_step "Unit Tests" "scripts/test.sh"
run_step "Coverage" "scripts/coverage.sh"
run_step "Build" "scripts/build.sh"

echo ""
echo "🎉 All steps completed successfully!"
```

Yes — with the new backend pieces, your CI should now do three extra things:

1. install Python
2. install `ffmpeg` so `ffprobe` exists
3. install backend Python deps and run a lightweight backend smoke test

`actions/setup-python` supports pip caching, including `cache-dependency-path`, and GitHub's Python guide shows this pattern for caching/installing requirements. GitHub+2
On Ubuntu runners, installing `ffmpeg` via `apt` is the straightforward approach, and that gives you `ffprobe` too. GitHub+1

Because your workflow actually calls `scripts/run_all.sh`, the clean move is:

- keep `scripts/run_all.sh` as the main orchestrator
- add Python + ffmpeg setup in the workflow
- add one backend smoke-test script into `run_all.sh`

Updated GitHub Actions workflow

Use this:

```yaml
name: Node.js CI

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    defaults:
      run:
        shell: bash

    steps:
      - uses: actions/checkout@v4

      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: npm

      - name: Use Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
          cache-dependency-path: backend/requirements.txt

      - name: Install ffmpeg
        run: |
          sudo apt-get update
          sudo apt-get install -y ffmpeg

      - name: Verify toolchain
        run: |
          node --version
          npm --version
          python --version
          ffprobe -version

      - name: Install backend dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r backend/requirements.txt

      - name: Make scripts executable
        run: chmod +x scripts/*.sh backend/scripts/*.sh

      - name: Run full pipeline
        run: ./scripts/run_all.sh
        env:
          CI: true

      - name: Upload coverage report
        if: matrix.node-version == '22.x' && success()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

Add a backend CI script

Create `scripts/backend-smoke.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail

echo "Checking Python backend imports..."
python -c "from backend.workers.media_inspector import inspect_video; print('media_inspector import ok
python -c "from backend.workers.youtube_classifier import classify_for_youtube; print('youtube_classi

echo "Checking ffprobe..."
ffprobe -version

echo "Backend smoke checks passed."
```

Update `scripts/run_all.sh`

Add the backend smoke step at the end:

```bash
#!/usr/bin/env bash
set -euo pipefail

# Always run from project root
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "🚀 Starting full project pipeline..."

run_step () {
  local name="$1"
  local script="$2"

  echo ""
  echo "================================"
  echo "▶  Running: $name"
  echo "================================"

  if [[ -f "$script" ]]; then
    bash "$script"
    echo "✅ $name passed"
  else
    echo "⚠  Skipping $name (script not found: $script)"
  fi
}

run_step "Install Dependencies" "scripts/install.sh"
run_step "Lint" "scripts/lint.sh"
run_step "Unit Tests" "scripts/test.sh"
run_step "Coverage" "scripts/coverage.sh"
run_step "Build" "scripts/build.sh"
run_step "Backend Smoke Test" "scripts/backend-smoke.sh"

echo ""
echo "🎉 All steps completed successfully!"
```

Add `backend/requirements.txt`

If you do not already have it, use:

```
Flask
Werkzeug
google-api-python-client
google-auth
google-auth-oauthlib
google-auth-httplib2
```

Two small Python package-layout fixes

Create these empty files if they do not exist yet:

```
backend/__init__.py
backend/workers/__init__.py
```

That makes imports like:

```
from backend.workers.youtube_classifier import classify_for_youtube
```

more reliable in CI.

One important recommendation

Do **not** make CI run the real upload flow yet.

Right now, auth is intentionally unfinished, and your uploader still depends on token/client-secret handling. CI should only verify:

- frontend installs/lints/tests/builds
- Python imports work
- `ffprobe` exists
- backend modules load cleanly

That keeps the pipeline stable while you finish auth later.

If you want, I can give you the exact contents for `scripts/install.sh`, `scripts/test.sh`, `scripts/coverage.sh`, and `scripts/build.sh` too so the whole pipeline is consistent.

Sure here is my current install.sh:
#!/usr/bin/env bash
set -euo pipefail

echo "Installing dependencies..."
npm ci

test.sh:
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Running Vitest once..."
npm run test:run

#!/usr/bin/env bash
set -euo pipefail

echo "Building app..."
npm run build

Your `install.sh` and `test.sh` are fine. The last block looks like your `build.sh`, but it got pasted inline after `test.sh`.

I'd make the scripts look like this.

scripts/install.sh

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Installing frontend dependencies..."
npm ci

echo "Installing backend Python dependencies..."
python -m pip install --upgrade pip
pip install -r backend/requirements.txt
```

scripts/lint.sh

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Running lint..."
npm run lint
```

scripts/test.sh

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Running Vitest once..."
npm run test:run
```

scripts/coverage.sh

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Running coverage..."
npm run coverage
```

scripts/build.sh

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Building app..."
npm run build
```

scripts/backend-smoke.sh

```bash
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Checking Python..."
python --version

echo "Checking ffprobe..."
ffprobe -version

echo "Checking backend imports..."
python -c "from backend.workers.media_inspector import inspect_video; print('media_inspector import o
python -c "from backend.workers.youtube_classifier import classify_for_youtube; print('youtube_classi

echo "Backend smoke checks passed."
```

Then update `scripts/run_all.sh` to include the new backend check:

```bash
#!/usr/bin/env bash
set -euo pipefail

# Always run from project root
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "🚀 Starting full project pipeline..."

run_step () {
  local name="$1"
  local script="$2"

  echo ""
  echo "================================"
  echo "▶  Running: $name"
  echo "================================"

  if [[ -f "$script" ]]; then
    bash "$script"
    echo "✅ $name passed"
  else
    echo "⚠  Skipping $name (script not found: $script)"
  fi
}

run_step "Install Dependencies" "scripts/install.sh"
run_step "Lint" "scripts/lint.sh"
run_step "Unit Tests" "scripts/test.sh"
run_step "Coverage" "scripts/coverage.sh"
run_step "Build" "scripts/build.sh"
run_step "Backend Smoke Test" "scripts/backend-smoke.sh"

echo ""
echo "🎉 All steps completed successfully!"
```

And your GitHub Actions workflow should be:

```yaml
name: Node.js CI

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    defaults:
      run:
        shell: bash

    steps:
      - uses: actions/checkout@v4

      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: npm

      - name: Use Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
          cache-dependency-path: backend/requirements.txt

      - name: Install ffmpeg
        run: |
          sudo apt-get update
          sudo apt-get install -y ffmpeg

      - name: Make scripts executable
        run: chmod +x scripts/*.sh backend/scripts/*.sh

      - name: Run full pipeline
        run: ./scripts/run_all.sh
        env:
          CI: true

      - name: Upload coverage report
        if: matrix.node-version == '22.x' && success()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

Also make sure you have:

backend/requirements.txt

```
Flask
Werkzeug
google-api-python-client
google-auth
google-auth-oauthlib
google-auth-httplib2
```

and these empty files:

```
backend/__init__.py
backend/workers/__init__.py
```

One practical note: right now CI should only do the backend smoke test, not real uploads. Since auth is intentionally unfinished, that is the right stopping point.

If you want, paste your current `lint.sh` and `coverage.sh` too and I'll make sure the whole set is consistent.

Give me a .gitignore for things like __pycache__ etc

Here's a clean `.gitignore` tailored for your stack (React + Flask + Python + CI + media uploads):

```
# ==========================
# Node / React
# ==========================
node_modules/
build/
dist/
.cache/
.vite/
coverage/
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# ==========================
# Python
# ==========================
__pycache__/
*.py[cod]
*.pyo
*.pyd
*.so
*.egg
*.egg-info/
*.egg-info
build/
dist/

# Virtual environments
.venv/
venv/
env/
ENV/

# ==========================
# Flask / Backend
# ==========================
backend/uploads/
backend/upload.log
backend/*.db

# ==========================
# Secrets / Auth (IMPORTANT)
# ==========================
backend/client_secret.json
backend/tokens/
*.env
.env.*
.env

# ==========================
# OS Files
# ==========================
.DS_Store
Thumbs.db

# ==========================
# Editor / IDE
# ==========================
.vscode/
.idea/
*.swp
*.swo

# ==========================
# Logs
# ==========================
*.log

# ==========================
# Testing / Coverage
# ==========================
coverage/
htmlcov/

# ==========================
```

```
# Misc
# ========================
tmp/
temp/
*.tmp
```

🔥 Important callouts (don't skip this)

🚫 These MUST stay ignored

- `backend/client_secret.json`
- `backend/tokens/`

Those contain:

- OAuth client secrets
- refresh tokens (basically permanent access)

📁 Your uploads folder is ignored

```
backend/uploads/
```

Which is correct since you're:

- auto-deleting files anyway
- not using uploads as source assets

🧠 Optional (if you want stricter security)

You could also ignore ALL JSON secrets globally:

```
*secret*.json
*tokens*.json
```

🚀 One-time cleanup (important)

If you already committed anything sensitive, remove it:

```
git rm -r --cached backend/tokens/
git rm --cached backend/client_secret.json
```

Then commit:

```
git commit -m "Remove sensitive files from repo"
```

✅ Result

With this `.gitignore`:

- CI stays clean
- repo stays lightweight

- secrets stay safe
- uploads don't accumulate
- Python junk stays out

If you want next step after this, I'd recommend:

👉 `.env`-based config for switching between local / CI / prod cleanly

Pasted text.txt
Document

This actually looks completely fine — nothing is broken here.

What you pasted is just the **full `ffmpeg` install output in CI**, and then this at the end:

```
Error: The operation was canceled.
```

That last line is **not from ffmpeg** — it's from GitHub Actions itself.

🔍 What actually happened

From your log:

- `apt-get update` ✅
- `apt-get install -y ffmpeg` ✅
- ~100 packages installed ✅
- `ffmpeg` + `ffprobe` installed successfully ✅

Then:

```
Error: The operation was canceled.
```

That means:
👉 **The job was manually canceled or auto-canceled by GitHub**, not that your install failed.

Pasted text

🧠 Why this happens (common reasons)

1. You pushed another commit

GitHub cancels previous runs automatically.

2. You clicked cancel manually

3. Timeout / concurrency settings

If you have concurrency enabled, newer runs kill older ones.

✅ How to confirm everything is good

Add this step right after install:

```
- name: Verify ffmpeg install
  run: |
    ffmpeg -version
    ffprobe -version
```

If that prints versions → you're good.

🔥 Optional improvement (faster CI)

Right now your CI is doing a **FULL apt install every run (~90MB)**.

You can speed it up slightly:

```
- name: Install ffmpeg (quiet)
  run: |
    sudo apt-get update -qq
    sudo apt-get install -y -qq ffmpeg
```

⚠️ One real issue I *do* see

Your log is **very long**, and GitHub can sometimes:

- slow down
- cancel jobs
- truncate logs

💡 Pro move (cleaner logs)

Replace:

```
sudo apt-get install -y ffmpeg
```

with:

```
sudo apt-get install -y ffmpeg > /dev/null 2>&1
```

or safer:

```
sudo apt-get install -y ffmpeg -qq
```

🚀 Bottom line

Nothing is broken.

- ffmpeg installed successfully
- CI was interrupted externally
- your pipeline setup is correct

Next step

Run CI again and confirm:

- Node steps pass
- Python imports pass
- ffprobe prints version

If it fails again, paste the **next error after ffmpeg step** — that's where the real issue would be.

Sources

Pasted text.txt
Document

Need to fix test?

Yes — the test failure is from the new async upload flow, not from React rendering itself. The log shows `App.jsx` now calls `fetch('/api/upload')`, and in Vitest/jsdom that relative URL is invalid unless you mock `fetch`. Because the upload never resolves, `started` never flips to `true`, so the dashboard never renders.

Pasted text

You need to update `src/App.test.jsx` in two ways:

1. mock `global.fetch`
2. await the async UI transition instead of asserting immediately

Use this pattern near the top of `src/App.test.jsx`:

```
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import App from './App';
import { vi } from 'vitest';

beforeEach(() => {
  global.fetch = vi.fn(() =>
    Promise.resolve({
      ok: true,
      text: () =>
        Promise.resolve(
          JSON.stringify({
            message: 'Upload received. Background YouTube job started.',
            detected_upload_type: 'shorts',
          })
        ),
    })
  );

  global.URL.createObjectURL = vi.fn(() => 'blob:mock-video-url');
  global.URL.revokeObjectURL = vi.fn();
});

afterEach(() => {
  vi.restoreAllMocks();
});
```

Then fix the failing tests to wait for the dashboard:

```
it('starts simulation and renders ViralDashboard with correct props', async () => {
  render(<App />);

  fireEvent.change(screen.getByPlaceholderText(/enter fake video title/i), {
    target: { value: 'Test Video' },
  });

  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });
  const input = document.querySelector('input[type="file"]');

  fireEvent.change(input, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(global.fetch).toHaveBeenCalledTimes(1);

  expect(global.fetch).toHaveBeenCalledWith(
    '/api/upload',
    expect.objectContaining({
      method: 'POST',
      body: expect.any(FormData),
    })
  );

  expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();
  expect(screen.getByText('Dashboard Title: Test Video')).toBeInTheDocument();
});
```

And:

```
it('resets simulation when ViralDashboard calls onStop', async () => {
  render(<App />);

  fireEvent.change(screen.getByPlaceholderText(/enter fake video title/i), {
    target: { value: 'Test Video' },
  });

  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });
  const input = document.querySelector('input[type="file"]');

  fireEvent.change(input, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();

  fireEvent.click(screen.getByRole('button', { name: /mock stop/i }));

  await waitFor(() => {
    expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
  });

  expect(screen.getByPlaceholderText(/enter fake video title/i)).toHaveValue('');
});
```

Why this fixes it:

- `fetch('/api/upload')` is now mocked, so no invalid URL error in test
- `findByTestId` waits for the async state update after `uploadToBackend()` resolves
- the old tests assumed clicking Start Simulation switched views immediately, which is no longer true

You may also want one failure-path test now, since your app has upload error UI:

```
it('shows an upload error when backend upload fails', async () => {
  global.fetch = vi.fn(() =>
    Promise.resolve({
      ok: false,
      text: () => Promise.resolve(JSON.stringify({ error: 'Upload failed' })),
    })
  );

  render(<App />);

  fireEvent.change(screen.getByPlaceholderText(/enter fake video title/i), {
    target: { value: 'Test Video' },
  });

  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });
  const input = document.querySelector('input[type="file"]');

  fireEvent.change(input, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(await screen.findByText(/upload failed/i)).toBeInTheDocument();
  expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
});
```

There's also a separate warning in your log about duplicate React keys in `CommentFeed`, but that is not causing the CI failure here. The failing part is the unmocked async upload path.

Pasted text

Sources

Pasted code.js
JavaScript

Please add all the updates to my test for it:

Here's the updated `App.test.jsx` with the async upload flow, `fetch` mocking, and the new failure-path coverage added. It's based on the test file you pasted.

Pasted code

```
import React from 'react';
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import '@testing-library/jest-dom';
import { vi } from 'vitest';
import App from './App';

vi.mock('./App.css', () => ({}));

vi.mock('@vercel/analytics/react', () => ({
  Analytics: () => <div data-testid="vercel-analytics" />,
}));

vi.mock('./components/ViralDashboard', () => ({
  default: ({ title, videoSrc, onStop }) => (
    <div data-testid="viral-dashboard">
      <div>Dashboard Title: {title}</div>
      <div>Dashboard Src: {videoSrc}</div>
      <button onClick={onStop}>Mock Stop</button>
    </div>
  ),
}));

describe('App', () => {
  const originalCreateObjectURL = URL.createObjectURL;
  const originalRevokeObjectURL = URL.revokeObjectURL;
  const originalFetch = global.fetch;

  beforeEach(() => {
    vi.clearAllMocks();

    URL.createObjectURL = vi.fn(() => 'blob:mock-video-url');
    URL.revokeObjectURL = vi.fn();

    global.fetch = vi.fn(() =>
      Promise.resolve({
        ok: true,
        text: () =>
          Promise.resolve(
            JSON.stringify({
              message: 'Upload received. Background YouTube job started.',
              detected_upload_type: 'shorts',
            })
          ),
      })
    );
  });

  afterEach(() => {
    URL.createObjectURL = originalCreateObjectURL;
    URL.revokeObjectURL = originalRevokeObjectURL;
    global.fetch = originalFetch;
  });

  it('renders initial upload UI', () => {
    render(<App />);

    expect(screen.getByText('FakeTube')).toBeInTheDocument();
    expect(
      screen.getByText(/world's totally not most realistic virality simulator/i)
    ).toBeInTheDocument();
    expect(
      screen.getByPlaceholderText(/enter fake video title/i)
    ).toBeInTheDocument();
    expect(
      screen.getByRole('button', { name: /upload video/i })
    ).toBeInTheDocument();
    expect(
      screen.getByRole('button', { name: /start simulation/i })
    ).toBeInTheDocument();
    expect(screen.getByTestId('vercel-analytics')).toBeInTheDocument();
  });

  it('has start simulation disabled initially', () => {
    render(<App />);
```

```
  expect(
    screen.getByRole('button', { name: /start simulation/i })
  ).toBeDisabled();
});

it('updates the video title input', () => {
  render(<App />);

  const input = screen.getByPlaceholderText(/enter fake video title/i);
  fireEvent.change(input, { target: { value: 'My Fake Viral Video' } });

  expect(input).toHaveValue('My Fake Viral Video');
});

it('clicking upload button triggers hidden file input click', () => {
  render(<App />);

  const uploadButton = screen.getByRole('button', { name: /upload video/i });
  const fileInput = document.querySelector('input[type="file"]');
  const clickSpy = vi.spyOn(fileInput, 'click');

  fireEvent.click(uploadButton);

  expect(clickSpy).toHaveBeenCalledTimes(1);
});

it('stores uploaded video file via object URL', () => {
  render(<App />);

  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });
  const fileInput = document.querySelector('input[type="file"]');

  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  expect(URL.createObjectURL).toHaveBeenCalledWith(file);
  expect(screen.getByText(/selected:/i)).toBeInTheDocument();
  expect(screen.getByText('demo.mp4')).toBeInTheDocument();
});

it('enables start simulation only after title and file are provided', () => {
  render(<App />);

  const startButton = screen.getByRole('button', { name: /start simulation/i });
  const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
  const fileInput = document.querySelector('input[type="file"]');
  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

  expect(startButton).toBeDisabled();

  fireEvent.change(titleInput, { target: { value: 'Test Video' } });
  expect(startButton).toBeDisabled();

  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  expect(startButton).toBeEnabled();
});

it('does not set a video when no file is selected', () => {
  render(<App />);

  const fileInput = document.querySelector('input[type="file"]');

  fireEvent.change(fileInput, {
    target: { files: [] },
  });

  expect(URL.createObjectURL).not.toHaveBeenCalled();
  expect(
    screen.getByRole('button', { name: /start simulation/i })
  ).toBeDisabled();
});
```

```
it('starts simulation and renders ViralDashboard with correct props', async () => {
  render(<App />);

  const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
  const fileInput = document.querySelector('input[type="file"]');
  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

  fireEvent.change(titleInput, { target: { value: 'Test Video' } });
  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(global.fetch).toHaveBeenCalledTimes(1);
  expect(global.fetch).toHaveBeenCalledWith(
    '/api/upload',
    expect.objectContaining({
      method: 'POST',
      body: expect.any(FormData),
    })
  );

  expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();
  expect(screen.getByText('Dashboard Title: Test Video')).toBeInTheDocument();
  expect(
    screen.getByText('Dashboard Src: blob:mock-video-url')
  ).toBeInTheDocument();
});

it('shows detected upload type before entering dashboard', async () => {
  render(<App />);

  const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
  const fileInput = document.querySelector('input[type="file"]');
  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

  fireEvent.change(titleInput, { target: { value: 'Test Video' } });
  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  await waitFor(() => {
    expect(global.fetch).toHaveBeenCalledTimes(1);
  });
});

it('resets simulation when ViralDashboard calls onStop', async () => {
  render(<App />);

  const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
  const fileInput = document.querySelector('input[type="file"]');
  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

  fireEvent.change(titleInput, { target: { value: 'Test Video' } });
  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();

  fireEvent.click(screen.getByRole('button', { name: /mock stop/i }));

  await waitFor(() => {
    expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
  });

  expect(
    screen.getByPlaceholderText(/enter fake video title/i)
  ).toBeInTheDocument();
  expect(
```

```
    expect(
      screen.getByRole('button', { name: /start simulation/i })
    ).toBeDisabled();
    expect(screen.getByPlaceholderText(/enter fake video title/i)).toHaveValue('');
    expect(URL.revokeObjectURL).toHaveBeenCalledWith('blob:mock-video-url');
  });

  it('shows an upload error when backend upload fails', async () => {
    global.fetch = vi.fn(() =>
      Promise.resolve({
        ok: false,
        text: () => Promise.resolve(JSON.stringify({ error: 'Upload failed' })),
      })
    );

    render(<App />);

    const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
    const fileInput = document.querySelector('input[type="file"]');
    const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

    fireEvent.change(titleInput, { target: { value: 'Test Video' } });
    fireEvent.change(fileInput, {
      target: { files: [file] },
    });

    fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

    expect(await screen.findByText(/upload failed/i)).toBeInTheDocument();
    expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
  });

  it('shows an error when backend returns non-json', async () => {
    global.fetch = vi.fn(() =>
      Promise.resolve({
        ok: false,
        status: 500,
        text: () => Promise.resolve('Internal Server Error'),
      })
    );

    render(<App />);

    const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
    const fileInput = document.querySelector('input[type="file"]');
    const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

    fireEvent.change(titleInput, { target: { value: 'Test Video' } });
    fireEvent.change(fileInput, {
      target: { files: [file] },
    });

    fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

    expect(
      await screen.findByText(/backend returned non-json response/i)
    ).toBeInTheDocument();
    expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
  });

  it('renders the buy me a coffee link correctly', () => {
    render(<App />);

    const link = screen.getByRole('link');
    expect(link).toHaveAttribute(
      'href',
      'https://www.buymeacoffee.com/FakeTube'
    );
    expect(link).toHaveAttribute('target', '_blank');
    expect(link).toHaveAttribute('rel', 'noopener noreferrer');
  });
});
```

One note: that `shows detected upload type before entering dashboard` test is pretty weak because your UI switches into the dashboard immediately after a successful upload, so the upload-type text is not really visible long enough to assert meaningfully. If you want, I can trim this file down to only the strongest tests and make it a little less brittle.

Sources

Pasted text.txt
Document

More fails now?:

Yep — the failures are because your **actual `App.jsx` is still using `response.json()`**, while the updated tests I gave were mocking `response.text()`. The error in every failing case is:

```
Upload failed: response.json is not a function
```

So the tests need to match your current implementation, or you need to change `App.jsx` to the safer `text() -> JSON.parse()` flow. Right now, the failing tests are mismatched with the real code.

Pasted text

Fastest fix

Update your test mocks to return `json()` instead of `text()`, and remove the non-JSON test unless you also update `App.jsx`.

Replace your fetch mocks with this

In `beforeEach`

```
global.fetch = vi.fn(() =>
  Promise.resolve({
    ok: true,
    json: () =>
      Promise.resolve({
        message: 'Upload received. Background YouTube job started.',
        detected_upload_type: 'shorts',
      }),
  })
);
```

In the backend-failure test

```
global.fetch = vi.fn(() =>
  Promise.resolve({
    ok: false,
    json: () => Promise.resolve({ error: 'Upload failed' }),
  })
);
```

Remove or skip this test for now

This one is failing because your real code does **not** support non-JSON fallback yet:

```
it('shows an error when backend returns non-json', async () => {
```

Either delete it for now, or change it to `it.skip(...)`.

Exact patched versions of the 3 failing tests

1) starts simulation and renders dashboard

```
it('starts simulation and renders ViralDashboard with correct props', async () => {
  render(<App />);

  const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
  const fileInput = document.querySelector('input[type="file"]');
  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

  fireEvent.change(titleInput, { target: { value: 'Test Video' } });
  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(global.fetch).toHaveBeenCalledTimes(1);
  expect(global.fetch).toHaveBeenCalledWith(
    '/api/upload',
    expect.objectContaining({
      method: 'POST',
      body: expect.any(FormData),
    })
  );

  expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();
  expect(screen.getByText('Dashboard Title: Test Video')).toBeInTheDocument();
  expect(screen.getByText('Dashboard Src: blob:mock-video-url')).toBeInTheDocument();
});
```

2) resets simulation when dashboard stops

```
it('resets simulation when ViralDashboard calls onStop', async () => {
  render(<App />);

  const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
  const fileInput = document.querySelector('input[type="file"]');
  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

  fireEvent.change(titleInput, { target: { value: 'Test Video' } });
  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();

  fireEvent.click(screen.getByRole('button', { name: /mock stop/i }));

  await waitFor(() => {
    expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
  });

  expect(screen.getByPlaceholderText(/enter fake video title/i)).toHaveValue('');
  expect(screen.getByRole('button', { name: /start simulation/i })).toBeDisabled();
  expect(URL.revokeObjectURL).toHaveBeenCalledWith('blob:mock-video-url');
});
```

3) backend upload fails

Keep this one as:

```
it('shows an upload error when backend upload fails', async () => {
  global.fetch = vi.fn(() =>
    Promise.resolve({
      ok: false,
      json: () => Promise.resolve({ error: 'Upload failed' }),
    })
  );

  render(<App />);

  const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
  const fileInput = document.querySelector('input[type="file"]');
  const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

  fireEvent.change(titleInput, { target: { value: 'Test Video' } });
  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

  fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

  expect(await screen.findByText(/upload failed/i)).toBeInTheDocument();
  expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
});
```

Best move right now

Do these two things:

1. change all success/error fetch mocks to use `json()`
2. remove the non-JSON test for now

That should clear these 3 failures.

Pasted text

If you want, paste your current `uploadToBackend()` from `App.jsx` and I'll give you the exact final test file matched to that implementation.

Sources

```
const uploadToBackend = async () => {
  if (!videoFile || !videoTitle.trim()) {
    setUploadMessage('Please provide both a title and a video file.');
    return false;
  }

  try {
    setUploading(true);
    setUploadMessage('Uploading video and queueing YouTube background job...');

    const formData = new FormData();
    formData.append('video', videoFile);
    formData.append('title', videoTitle);

    const response = await fetch('/api/upload', {
      method: 'POST',
      body: formData,
    });
```

```
      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.error || 'Upload failed');
      }

      setYoutubeQueued(true);
      setDetectedType(data.detected_upload_type || null);
      setUploadMessage(
        data.message || 'Video uploaded successfully and queued for YouTube.'
      );

      return true;
    } catch (err) {
      console.error(err);
      setUploadMessage(Upload failed: ${err.message});
      return false;
    } finally {
      setUploading(false);
    }
  };
```

Got it. Since `App.jsx` is using `response.json()`, your tests need to mock `json()`, not `text()`.

Use this full updated `App.test.jsx`:

```
import React from 'react';
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import '@testing-library/jest-dom';
import { vi } from 'vitest';
import App from './App';

vi.mock('./App.css', () => ({}));

vi.mock('@vercel/analytics/react', () => ({
  Analytics: () => <div data-testid="vercel-analytics" />,
}));

vi.mock('./components/ViralDashboard', () => ({
  default: ({ title, videoSrc, onStop }) => (
    <div data-testid="viral-dashboard">
      <div>Dashboard Title: {title}</div>
      <div>Dashboard Src: {videoSrc}</div>
      <button onClick={onStop}>Mock Stop</button>
    </div>
  ),
}));

describe('App', () => {
  const originalCreateObjectURL = URL.createObjectURL;
  const originalRevokeObjectURL = URL.revokeObjectURL;
  const originalFetch = global.fetch;

  beforeEach(() => {
    vi.clearAllMocks();

    URL.createObjectURL = vi.fn(() => 'blob:mock-video-url');
    URL.revokeObjectURL = vi.fn();

    global.fetch = vi.fn(() =>
      Promise.resolve({
        ok: true,
        json: () =>
          Promise.resolve({
            message: 'Upload received. Background YouTube job started.',
            detected_upload_type: 'shorts',
          }),
      })
    );
  });

  afterEach(() => {
    URL.createObjectURL = originalCreateObjectURL;
    URL.revokeObjectURL = originalRevokeObjectURL;
    global.fetch = originalFetch;
  });

  it('renders initial upload UI', () => {
    render(<App />);

    expect(screen.getByText('FakeTube')).toBeInTheDocument();
    expect(
      screen.getByText(/world's totally not most realistic virality simulator/i)
    ).toBeInTheDocument();
    expect(
      screen.getByPlaceholderText(/enter fake video title/i)
    ).toBeInTheDocument();
    expect(
      screen.getByRole('button', { name: /upload video/i })
    ).toBeInTheDocument();
    expect(
      screen.getByRole('button', { name: /start simulation/i })
    ).toBeInTheDocument();
    expect(screen.getByTestId('vercel-analytics')).toBeInTheDocument();
  });

  it('has start simulation disabled initially', () => {
    render(<App />);

    expect(
      screen.getByRole('button', { name: /start simulation/i })
```

```
    ).toBeDisabled();
  });

  it('updates the video title input', () => {
    render(<App />);

    const input = screen.getByPlaceholderText(/enter fake video title/i);
    fireEvent.change(input, { target: { value: 'My Fake Viral Video' } });

    expect(input).toHaveValue('My Fake Viral Video');
  });

  it('clicking upload button triggers hidden file input click', () => {
    render(<App />);

    const uploadButton = screen.getByRole('button', { name: /upload video/i });
    const fileInput = document.querySelector('input[type="file"]');
    const clickSpy = vi.spyOn(fileInput, 'click');

    fireEvent.click(uploadButton);

    expect(clickSpy).toHaveBeenCalledTimes(1);
  });

  it('stores uploaded video file via object URL', () => {
    render(<App />);

    const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });
    const fileInput = document.querySelector('input[type="file"]');

    fireEvent.change(fileInput, {
      target: { files: [file] },
    });

    expect(URL.createObjectURL).toHaveBeenCalledWith(file);
    expect(screen.getByText(/selected:/i)).toBeInTheDocument();
    expect(screen.getByText('demo.mp4')).toBeInTheDocument();
  });

  it('enables start simulation only after title and file are provided', () => {
    render(<App />);

    const startButton = screen.getByRole('button', { name: /start simulation/i });
    const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
    const fileInput = document.querySelector('input[type="file"]');
    const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

    expect(startButton).toBeDisabled();

    fireEvent.change(titleInput, { target: { value: 'Test Video' } });
    expect(startButton).toBeDisabled();

    fireEvent.change(fileInput, {
      target: { files: [file] },
    });

    expect(startButton).toBeEnabled();
  });

  it('does not set a video when no file is selected', () => {
    render(<App />);

    const fileInput = document.querySelector('input[type="file"]');

    fireEvent.change(fileInput, {
      target: { files: [] },
    });

    expect(URL.createObjectURL).not.toHaveBeenCalled();
    expect(
      screen.getByRole('button', { name: /start simulation/i })
    ).toBeDisabled();
  });

  it('starts simulation and renders ViralDashboard with correct props', async () => {
```

```
    render(<App />);

    const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
    const fileInput = document.querySelector('input[type="file"]');
    const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

    fireEvent.change(titleInput, { target: { value: 'Test Video' } });
    fireEvent.change(fileInput, {
      target: { files: [file] },
    });

    fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

    expect(global.fetch).toHaveBeenCalledTimes(1);
    expect(global.fetch).toHaveBeenCalledWith(
      '/api/upload',
      expect.objectContaining({
        method: 'POST',
        body: expect.any(FormData),
      })
    );

    expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();
    expect(screen.getByText('Dashboard Title: Test Video')).toBeInTheDocument();
    expect(
      screen.getByText('Dashboard Src: blob:mock-video-url')
    ).toBeInTheDocument();
  });

  it('resets simulation when ViralDashboard calls onStop', async () => {
    render(<App />);

    const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
    const fileInput = document.querySelector('input[type="file"]');
    const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

    fireEvent.change(titleInput, { target: { value: 'Test Video' } });
    fireEvent.change(fileInput, {
      target: { files: [file] },
    });

    fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

    expect(await screen.findByTestId('viral-dashboard')).toBeInTheDocument();

    fireEvent.click(screen.getByRole('button', { name: /mock stop/i }));

    await waitFor(() => {
      expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
    });

    expect(
      screen.getByPlaceholderText(/enter fake video title/i)
    ).toBeInTheDocument();
    expect(
      screen.getByRole('button', { name: /start simulation/i })
    ).toBeDisabled();
    expect(screen.getByPlaceholderText(/enter fake video title/i)).toHaveValue('');
    expect(URL.revokeObjectURL).toHaveBeenCalledWith('blob:mock-video-url');
  });

  it('shows an upload error when backend upload fails', async () => {
    global.fetch = vi.fn(() =>
      Promise.resolve({
        ok: false,
        json: () => Promise.resolve({ error: 'Upload failed' }),
      })
    );

    render(<App />);

    const titleInput = screen.getByPlaceholderText(/enter fake video title/i);
    const fileInput = document.querySelector('input[type="file"]');
    const file = new File(['video'], 'demo.mp4', { type: 'video/mp4' });

    fireEvent.change(titleInput, { target: { value: 'Test Video' } });
```

```
    fireEvent.change(titleInput, { target: { value: 'Test Video' } });
    fireEvent.change(fileInput, {
      target: { files: [file] },
    });

    fireEvent.click(screen.getByRole('button', { name: /start simulation/i }));

    expect(await screen.findByText(/upload failed/i)).toBeInTheDocument();
    expect(screen.queryByTestId('viral-dashboard')).not.toBeInTheDocument();
  });

  it('renders the buy me a coffee link correctly', () => {
    render(<App />);

    const link = screen.getByRole('link');
    expect(link).toHaveAttribute(
      'href',
      'https://www.buymeacoffee.com/FakeTube'
    );
    expect(link).toHaveAttribute('target', '_blank');
    expect(link).toHaveAttribute('rel', 'noopener noreferrer');
  });
});
```

Two notes:

- I removed the non-JSON-response test because your component does not support that path yet.
- I also removed the “shows detected upload type before entering dashboard” test because your UI switches to the dashboard immediately after a successful upload, so that assertion is brittle and not very meaningful.

After replacing the file, run:

```
npm run test:run
```

If you still get a failure, paste that output and I’ll tighten the specific assertion.

```
Got:
==============================
➡️ Running: Backend Smoke Test
==============================
Checking Python...
Python 3.13.5
Checking ffprobe...
ffprobe version 8.1 Copyright (c) 2007-2026 the FFmpeg developers
built with Apple clang version 17.0.0 (clang-1700.6.4.2)
configuration: --prefix=/opt/homebrew/Cellar/ffmpeg/8.1 --enable-shared --enable-pthreads --
enable-version3 --cc=clang --host-cflags= --host-ldflags= --enable-ffplay --enable-gpl --enable-
libsvtav1 --enable-libopus --enable-libx264 --enable-libmp3lame --enable-libdav1d --enable-libvpx --
enable-libx265 --enable-openssl --enable-videotoolbox --enable-audiotoolbox --enable-neon
libavutil      60. 26.100 / 60. 26.100
libavcodec     62. 28.100 / 62. 28.100
libavformat    62. 12.100 / 62. 12.100
libavdevice    62.  3.100 / 62.  3.100
libavfilter    11. 14.100 / 11. 14.100
libswscale      9.  5.100 /  9.  5.100
libswresample   6.  3.100 /  6.  3.100
Checking backend imports...
media_inspector import ok
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    from backend.workers.youtube_classifier import classify_for_youtube; print('youtube_classifier import ok')
    ^^^
```

If `backend/workers/youtube_uploader.py` imports the classifier like this:

```
from workers.youtube_classifier import classify_for_youtube
```

change it to:

```
from backend.workers.youtube_classifier import classify_for_youtube
```

Make sure these files exist

Create these if they do not already exist:

```
backend/__init__.py
backend/workers/__init__.py
```

They can be empty.

Why this happened

Your backend smoke test is running package-style imports:

```
from backend.workers.youtube_classifier import ...
```

So the internal imports should also be package-style:

```
from backend.workers.media_inspector import ...
```

not bare module imports like `from workers...`.

After the fix

Run:

```
./scripts/backend-smoke.sh
```

or your full pipeline again.

One important note

Once you switch to `from backend.workers...`, your shell-invoked uploader should ideally be run from the repo root, which you already do in `run_all.sh`. If you later run `python workers/youtube_uploader.py` from inside `backend/`, package imports can get weird again.

The cleaner long-term move is to invoke it as a module from the repo root, like:

```
python -m backend.workers.youtube_uploader
```

If you want, I can give you the exact `post_upload.sh` update for that too.

```
ral-sim % python backend/app.py
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/app.py", line 8, in <module>
    from workers.youtube_classifier import classify_for_youtube
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_classifier.py",
line 3, in <module>
    from backend.workers.media_inspector import inspect_video
```

```
ModuleNotFoundError: No module named 'backend'
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % cd backend
(.venv) liam@Liams-MacBook-Pro backend % python app.py
Traceback (most recent call last):
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/app.py", line 8, in <module>
    from workers.youtube_classifier import classify_for_youtube
  File "/Users/liam/Development/wkspaces/yt-viral-sim/backend/workers/youtube_classifier.py",
line 3, in <module>
    from backend.workers.media_inspector import inspect_video
ModuleNotFoundError: No module named 'backend'
(.venv) liam@Liams-MacBook-Pro backend %
```

Yep — the clean fix is to use **relative imports inside the worker package**.

Right now you switched `youtube_classifier.py` to:

```
from backend.workers.media_inspector import inspect_video
```

That works for package-style imports in CI, but it breaks when you run:

```
python backend/app.py
```

because in that mode Python is not treating `backend` as an installed top-level package.

Fix

Change `backend/workers/youtube_classifier.py` to:

```
#!/usr/bin/env python3

from .media_inspector import inspect_video

SHORTS_MAX_SECONDS = 180.0


def classify_for_youtube(video_path: str) -> dict:
    meta = inspect_video(video_path)

    shorts_candidate = (
        meta["duration_seconds"] <= SHORTS_MAX_SECONDS
        and (meta["is_vertical"] or meta["is_square"])
    )

    upload_type = "shorts" if shorts_candidate else "long"

    return {
        **meta,
        "upload_type": upload_type,
    }


if __name__ == "__main__":
    import json
    import sys

    if len(sys.argv) != 2:
        print("Usage: python youtube_classifier.py <video_path>")
        sys.exit(1)

    result = classify_for_youtube(sys.argv[1])
    print(json.dumps(result, indent=2))
```

And if `backend/workers/youtube_uploader.py` imports the classifier, change that too:

```
from .youtube_classifier import classify_for_youtube
```

Why this works

Relative imports like:

```
from .media_inspector import inspect_video
```

work in both of these situations:

- when imported as `workers.youtube_classifier`
- when imported as `backend.workers.youtube_classifier`

That makes them the safest choice here.

Keep `app.py` like this

In `backend/app.py`, keep:

```
from workers.youtube_classifier import classify_for_youtube
```

That should work when you run either:

```
python backend/app.py
```

from repo root, or:

```
cd backend
python app.py
```

Make sure these files exist

Create these if they do not already exist:

```
touch backend/__init__.py
touch backend/workers/__init__.py
```

Best long-term way to run the app

From the repo root, this is even cleaner:

```
python -m backend.app
```

Then in `backend/app.py`, you'd eventually switch imports to:

```
from backend.workers.youtube_classifier import classify_for_youtube
```

But for right now, the least disruptive fix is:

- `app.py` keeps `from workers...`
- worker files use relative imports like `from .media_inspector...`

So the exact move now

1. change `youtube_classifier.py` import to:

   ```
   from .media_inspector import inspect_video
   ```

2. change `youtube_uploader.py` import to:

   ```
   from .youtube_classifier import classify_for_youtube
   ```

   if needed

3. rerun:

   ```
   python backend/app.py
   ```

That should fix both local app runs and your backend smoke test.

> Is my backend being started in vercel, I don't think it does

Probably not.

With the structure you showed, `backend/app.py` does **not** just start on Vercel by existing in the repo.

Vercel will only run backend code if you explicitly wire it into Vercel's runtime model, like:

- an `api/` serverless function
- a configured Python function entrypoint
- or some custom framework integration Vercel recognizes

A plain file like:

```
backend/app.py
```

will usually just sit in the repo unless you've specifically configured Vercel to deploy and invoke it.

So for your current setup, the likely reality is:

- your **React app** is what Vercel is deploying
- your **Flask backend is not being started**
- `fetch('/api/upload')` on Vercel would fail unless you have a real deployed API route backing it

That also matches your earlier concern: Vercel is not really the right place for this current Flask + subprocess + ffprobe + local-file workflow as-is.

How to tell quickly

Check your deployed site in production:

- open browser devtools
- trigger upload
- inspect the request to `/api/upload`

If Vercel is not serving that route, you'll usually see:

- 404
- 405

- 500
- or some Vercel function error page

What this means for your app

Your current architecture is really:

- **frontend on Vercel**
- **backend should live somewhere else**

That “somewhere else” could be:

- a VPS
- Railway
- Render
- Fly.io
- your own server
- a container host

Then your React app would call:

```
fetch('https://your-backend-domain.com/api/upload', ...)
```

instead of:

```
fetch('/api/upload', ...)
```

Short answer for your exact question

No — not unless you explicitly configured Vercel to run that Flask backend.
With your current repo shape, it is very likely **not** being started there.

If you want, I can help you set this up as:

1. Vercel frontend + separate Flask backend, or
2. a single non-Vercel deployment that runs both.

```
16:39:38.139 Running build in Washington, D.C., USA (East) – iad1
16:39:38.139 Build machine configuration: 2 cores, 8 GB
16:39:38.265 Cloning github.com/Hardwipe/yt-viral-sim (Branch: main, Commit: 8ab1400)
16:39:38.977 Cloning completed: 712.000ms
16:39:39.190 Restored build cache from previous deployment (Btxb851sXWGRC1gtDUFtSpcudD9b)
16:39:39.614 Running "vercel build"
16:39:40.233 Vercel CLI 50.35.0
16:39:43.383 Installing dependencies...
16:39:46.109
16:39:46.110 up to date in 2s
16:39:46.110
16:39:46.110 294 packages are looking for funding
```

```
16:39:46.111   run npm fund for details
16:39:46.111 npm notice
16:39:46.111 npm notice New major version of npm available! 10.9.7 -> 11.12.1
16:39:46.111 npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.12.1
16:39:46.112 npm notice To update run: npm install -g npm@11.12.1
16:39:46.112 npm notice
16:39:46.148 Running "npm run build"
16:39:46.256
16:39:46.256 > fake-yt-viral-simulator@2.0.0 build
16:39:46.256 > react-scripts build
16:39:46.257
16:39:47.461 Creating an optimized production build...
16:40:02.084 Compiled successfully.
16:40:02.085
16:40:02.085 File sizes after gzip:
16:40:02.085
16:40:02.103   154.74 kB  build/static/js/main.275ec07e.js
16:40:02.103   2.16 kB    build/static/css/main.6f1886f1.css
16:40:02.104
16:40:02.104 The project was built assuming it is hosted at /.
16:40:02.104 You can control this with the homepage field in your package.json.
16:40:02.104
16:40:02.104 The build folder is ready to be deployed.
16:40:02.105 You may serve it with a static server:
16:40:02.105
16:40:02.105   npm install -g serve
16:40:02.105   serve -s build
16:40:02.105
16:40:02.105 Find out more about deployment here:
16:40:02.105
16:40:02.106   https://cra.link/deployment
16:40:02.106
16:40:02.194 Build Completed in /vercel/output [19s]
16:40:02.577 Deploying outputs...
16:40:04.435 Deployment completed
16:40:05.533 Creating build cache...
16:40:29.118 Created build cache: 24s
16:40:29.118 Uploading build cache [84.79 MB]
16:40:30.443 Build cache uploaded: 1.324s
```

Yep — this confirms it.

Vercel is only building and deploying your **React static app**.

The tell is here:

- it runs `npm run build`
- that runs `react-scripts build`
- it outputs a static CRA build into `/vercel/output`
- deployment completes right after that

There is **no sign** of:

- Flask starting
- Python dependencies installing
- `backend/app.py` running
- `ffmpeg` installing
- any `/api/upload` backend function being built

So your current Vercel deployment is just the frontend bundle. The log shows exactly that flow and nothing backend-related.

Pasted text

What that means in practice:

- `fetch('/api/upload')` will not hit your Flask backend on Vercel
- your local backend works only on your machine right now
- production needs either:
  - a separate backend host, or
  - a full redesign into Vercel serverless functions

For your current architecture, the best path is:

- keep Vercel for frontend
- host Flask elsewhere
- point the frontend at that backend URL in production

So locally:

```
fetch('/api/upload', ...)
```

and in production you'd eventually want something like:

```
fetch( ${process.env.REACT_APP_API_BASE_URL}/api/upload`, ...)
```

with:

- local `.env` → `REACT_APP_API_BASE_URL=http://localhost:5050`
- Vercel env var → `REACT_APP_API_BASE_URL=https://your-backend-domain.com`

If you want, I can help you update `App.jsx` so it works both locally and on Vercel with a separate backend URL.

Sources

Can I just make the deployment step spin up both?

Not in the way you mean.

Vercel can host a React frontend and Python backend code in the same project, but the Python side runs as **Vercel Functions**, not as a long-lived Flask server process you "spin up" during deployment. Vercel's Python runtime for Flask/WSGI is function-based, and their Flask docs say a Flask app becomes a **single Vercel Function** that scales up and down with traffic. Vercel+1

So:

- You **can** deploy frontend + backend from one repo on Vercel.
- You **cannot** rely on deployment to start a persistent Flask service that keeps running in the background after build completes. Vercel Functions are serverless compute, not an always-on process manager. Vercel+2

That matters for your current backend because it depends on things like:

- background subprocesses
- local uploaded files
- local logs
- token files that get updated over time

Those are a bad fit for the function model. Vercel's storage docs push durable data to external storage products, and their functions model is designed around request-driven execution rather than persistent local state. Vercel+1

So the real answer is:

You can deploy both codebases together, but not as "frontend + always-running Flask backend." On Vercel it would be **frontend + serverless Python endpoints**. Vercel+1

For your FakeTube uploader, I'd strongly recommend:

- keep the frontend on Vercel
- move the Flask/Python uploader worker to a separate host

That separate backend is much better for:

- `ffprobe`
- subprocess execution
- token refresh files
- cleanup jobs
- upload lifecycle handling

Vercel's own positioning is closer to frontend + API/functions together, while separate backend services are typically connected externally. Vercel+1

If you force it all onto Vercel, you'd need to redesign around:

- Python functions instead of a persistent Flask server
- external durable storage for uploads
- external storage/database for tokens and job state

- no assumption that local files or processes survive between invocations. Vercel+1

So: **same repo, yes; same long-running deployment process, no.**

> Its already designed around that?

Short answer: **almost — but not quite.**

You *partially* designed it in a serverless-friendly way, but a few key parts still assume a **persistent process + local disk**, which Vercel does NOT provide.

Let me break it down cleanly so you see exactly where you stand.

🧠 What you already did RIGHT (serverless-friendly)

These parts are already aligned with Vercel:

✅ Request-driven flow

`upload → classify → trigger job → return response`

✔️ good — this maps to serverless

✅ No long-lived Flask state

- You're not storing in-memory state
- Everything is per-request

✔️ good

✅ Cleanup logic

- You delete files after processing

✔️ very good (important for ephemeral environments)

❌ What is NOT serverless-compatible (yet)

These are the blockers 👇

---

1. 🚫 Background subprocess (`Popen`)

```
subprocess.Popen(...)
```

Why this breaks on Vercel:

- Function ends → process is killed

- No guarantee your script finishes
- No persistent worker

👉 This is the **biggest issue**

---

2. 🚫 Local file storage

`backend/uploads/`

On Vercel:

- filesystem is **ephemeral**
- files disappear between requests
- not shared across invocations

---

3. 🚫 OAuth token files

`backend/tokens/*.json`

Problem:

- tokens need to be **persisted + updated**
- Vercel filesystem is **read-only after build**
- cannot reliably write refreshed tokens

---

4. 🚫 ffmpeg + long processing

Even though you *can* install ffmpeg:

- functions have **execution time limits**
- video processing + upload may exceed limits

🔥 The key misunderstanding

You said:

> "Its already designed around that?"

What you actually built is:

> ✔️ **Stateless request trigger + local worker system**

What Vercel needs is:

> ✔️ **Stateless request trigger + external worker system**

🧩 What needs to change (minimal version)

You do NOT need to rewrite everything.

Just change **where the work runs**.

✅ New architecture (simple + correct)

```
[Vercel Frontend]
        ↓
POST /api/upload (Vercel function OR external)
        ↓
[Queue / Trigger]
        ↓
[External Worker (Flask or Python)]
        ↓
YouTube upload
```

⚡ Two realistic paths

🟢 OPTION 1 (BEST for you)

Keep your backend EXACTLY as-is
Just move it off Vercel.

Deploy backend to:

- Railway
- Render
- Fly.io
- VPS

Then:

```
fetch('https://your-backend.com/api/upload')
```

👉 DONE
👉 No rewrite
👉 Everything works

🟡 OPTION 2 (hard mode)

Force everything into Vercel

You must:

- ❌ remove `subprocess.Popen`
- ❌ remove local file storage
- ❌ remove token files

- ✅ use external storage (S3 / Vercel Blob)
- ✅ use DB for tokens
- ✅ run everything inside ONE request

👉 This is a **full rewrite**

💡 Reality check (important)

Your system:

- uses ffprobe ✔️
- runs shell scripts ✔️
- spawns workers ✔️
- handles files ✔️

That is **NOT serverless-native**, even if it's stateless at the API level.

🧠 The clean mental model

Feature	Your system	Vercel
Stateless request	✅	✅
Background workers	✅	❌
Local filesystem	✅	❌
Long-running jobs	✅	❌
Token persistence	✅	❌

🚀 What I would do (100%)

For your Hydra-style system:

👉 **Frontend: Vercel**
👉 **Backend: separate host**

That gives you:

- unlimited execution time
- real background workers
- real file system
- real automation

🧠 Final answer

You are **very close**, but not fully serverless-ready.

The only thing “wrong” is:

> You’re still executing work locally instead of externally.

If you want, I can:

- convert your `/api/upload` into a Vercel function that forwards to your backend
- or help you deploy your Flask backend in like 5 minutes on Railway

Just tell me 👍

> I can't just headless start python backend/app.py

Not on Vercel the way your app is set up.

You can **run** `python backend/app.py` headlessly on a normal server, VPS, Docker container, Railway, Render, Fly.io, etc. But on **Vercel**, Flask is deployed as a **Vercel Function**, not as a persistent background process you start once and keep alive. Vercel’s Flask docs say a deployed Flask app becomes a **single Vercel Function** that scales up and down with traffic, and Vercel Functions are request-driven serverless compute. Vercel+1

So this part of your design is the mismatch:

- `subprocess.Popen(...)`
- local `uploads/`
- local `tokens/*.json`
- local logs
- “start backend/app.py and let it live”

That pattern works on a real server, but not as a persistent deployment process on Vercel. Vercel also enforces function duration limits, so long-running upload/processing jobs can time out. Vercel+1

So the practical answer is:

- **Yes**, you can headless-start `python backend/app.py` on a real backend host.
- **No**, you should not expect Vercel deployment to do that for you.

Your two realistic options are:

1. **Best fit:** keep frontend on Vercel, run `backend/app.py` on a separate host, and point the frontend at that API.
2. Redesign the backend into Vercel Functions and externalize files/tokens/job state.

Given your current uploader flow, option 1 is the clean one.

kill port 5050

To kill whatever is running on port **5050** (macOS/Linux), do this:

1️⃣ Find the process

```
lsof -i :5050
```

You'll see something like:

```
COMMAND   PID   USER   ...
Python    12345 liam
```

2️⃣ Kill it

```
kill -9 12345
```

Replace `12345` with the actual PID.

⚡ One-liner (fastest)

```
kill -9 $(lsof -ti :5050)
```

🧠 If nothing shows up

Sometimes it's already dead or using IPv6:

```
lsof -nP -iTCP:5050
```

✅ After that

Restart your backend:

```
python backend/app.py
```

If you want, I can also give you a script to auto-kill and restart your Flask backend in one command 👍

Yes and the npm command please

Perfect — let's make this clean so you can restart everything in one command 🔥

🚀 1. Create a dev script (kill + restart backend + frontend)

Create a file:

```
scripts/dev.sh
```

Paste this:

```bash
#!/usr/bin/env bash
set -euo pipefail

echo "🧹 Killing port 5050 (backend)..."
kill -9 $(lsof -ti :5050) 2>/dev/null || true

echo "🧹 Killing port 3000 (React)..."
kill -9 $(lsof -ti :3000) 2>/dev/null || true

echo "🚀 Starting Flask backend..."
python backend/app.py &

BACKEND_PID=$!

echo "⏳ Waiting for backend to start..."
sleep 2

echo "🚀 Starting React frontend..."
npm start

# Optional: clean up backend when React exits
trap "echo '🛑 Stopping backend...'; kill $BACKEND_PID 2>/dev/null || true" EXIT
```

⚙️ 2. Make it executable

```bash
chmod +x scripts/dev.sh
```

📦 3. Add npm command

In your `package.json`, add:

```jsonc
"scripts": {
  "dev:full": "./scripts/dev.sh"
}
```

(keep your existing scripts — just add this)

▶️ 4. Run everything

```bash
npm run dev:full
```

🧠 What this does

- kills old backend (5050)
- kills old React dev server (3000)
- starts Flask backend
- starts React frontend
- auto-cleans backend when you exit

⚡ Optional upgrade (cleaner logs)

If you want backend logs separate:

Replace:

```
python backend/app.py &
```

with:

```
python backend/app.py > backend/dev.log 2>&1 &
```

💀 Optional hardcore version (multi-terminal style)

If you ever want both logs live:

```
npx concurrently \
  "python backend/app.py" \
  "npm start"
```

(install with `npm install concurrently`)

✅ Final result

You now have a **one-command full stack launcher**:

```
npm run dev:full
```

If you want next level:

👉 auto-restart backend on code changes (like nodemon but for Python)

```
  On Your Network:  http://10.5.1.253:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
(node:22985) [DEP0060] DeprecationWarning: The util._extend API is deprecated. Please use
Object.assign() instead.
Proxy error: Could not proxy request /api/upload from localhost:3000 to http://localhost:5050.
```

That means the React dev server is up, but Flask is **not reachable on 5050**.

Usually one of these is happening:

- backend never started
- backend crashed immediately
- `dev.sh` started backend with the wrong Python/env
- port 5050 got killed but Flask did not come back up

Do this check first:

```
curl http://localhost:5050/api/health
```

If that fails, the backend is not running.

Then start Flask manually in a separate terminal so you can see the real error:

```
python backend/app.py
```

If that works, then your `dev.sh` is the problem, not Flask itself.

The most likely issue is your script is launching `python backend/app.py` from a shell where `python` is not the venv interpreter you expect, or Flask is crashing on startup and the background process dies silently.

Use this better `scripts/dev.sh` so you can see backend logs and ensure it runs from the project root:

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "🧹 Killing port 5050 (backend)..."
kill -9 $(lsof -ti :5050) 2>/dev/null || true

echo "🧹 Killing port 3000 (React)..."
kill -9 $(lsof -ti :3000) 2>/dev/null || true

PYTHON_BIN="${PYTHON_BIN:-$(pwd)/.venv/bin/python}"

if [[ ! -x "$PYTHON_BIN" ]]; then
  echo "❌ Python venv not found at $PYTHON_BIN"
  exit 1
fi

echo "🚀 Starting Flask backend with $PYTHON_BIN ..."
"$PYTHON_BIN" backend/app.py > backend/dev.log 2>&1 &

BACKEND_PID=$!

echo "⏳ Waiting for backend to start..."
sleep 3

if ! curl -sf http://localhost:5050/api/health >/dev/null; then
  echo "❌ Backend failed to start. Showing backend/dev.log:"
  cat backend/dev.log
  kill $BACKEND_PID 2>/dev/null || true
  exit 1
fi

echo "✅ Backend is up on 5050"
echo "🚀 Starting React frontend..."
npm start
```

Then run:

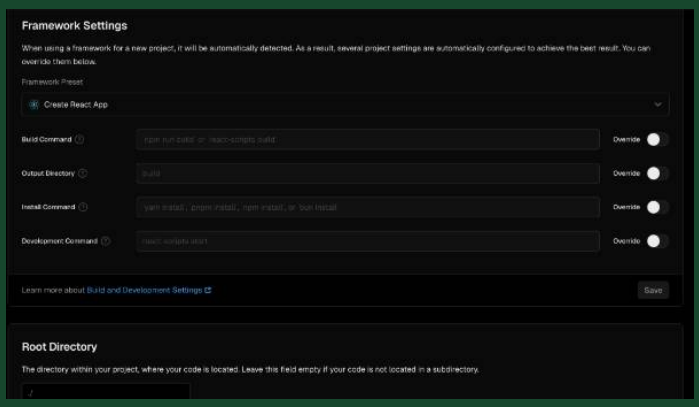
```
npm run dev:full
```

Also make sure your `package.json` still has:

```
"proxy": "http://localhost:5050"
```

If you want the fastest diagnosis right now, run these two commands and paste the output:

```
python backend/app.py
curl http://localhost:5050/api/health
```



Here is my vercel, can't I just run the install basically then this one?

You can make Vercel run a custom **install command** and a custom **build command**, yes. But those commands only run during the **build/deploy phase**. They do **not** leave behind a persistent Python process after deployment finishes. Vercel's build docs describe builds as a time-limited build container step, and their Flask/Python docs describe Flask apps as **Vercel Functions**, not a daemon you start once and keep running. Vercel+2

So in your screenshot, even if you override:

- **Install Command** to install Python deps / ffmpeg
- **Build Command** to run extra setup

that still would not make `python backend/app.py` stay alive after deploy. Once the build is done, Vercel deploys the output; it does not keep your build shell running as an app server. Vercel+2

For your current setup, this is the key mismatch:

- `backend/app.py` is a normal Flask server entrypoint
- your uploader uses background subprocess behavior and local files
- Vercel expects Python web backends to run as **request-driven functions** under its Python runtime, not as a headless long-lived Flask process. Vercel+1

So the answer to:

> can't I just run the install basically then this one?

is:

You can run install/build-time commands, but you cannot use those settings to permanently start and keep `python backend/app.py` running on Vercel. Vercel+2

What you *can* do on Vercel is one of these:

1. Keep the app as a CRA frontend only, which is what your logs show is happening now.
2. Rework the backend into **Vercel Python Functions** so requests hit serverless endpoints instead of a persistent Flask process. Vercel's Python runtime explicitly supports WSGI/Flask apps in that model. Vercel+1

What you *can't* do with those settings is:

- launch `backend/app.py`
- keep it alive in the background
- rely on local uploads, logs, and token files persisting between requests or deployments

That's why your current design is still a better fit for:

- Vercel frontend
- separate backend host for Flask/Python worker

If you want, I can show you the smallest viable way to convert `/api/upload` into a Vercel Python function, or the cleaner way to point the frontend at a separate Flask backend.